Apuntes – Clase 12

Swapping Mechanism

Sistemas Operativos

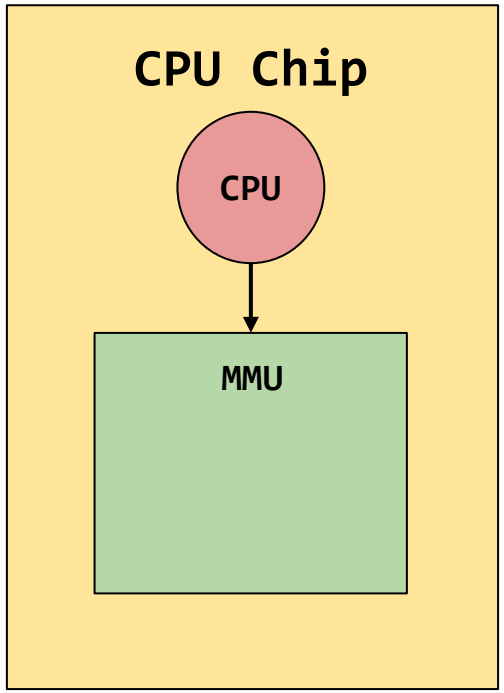
Universidad de Antioquia

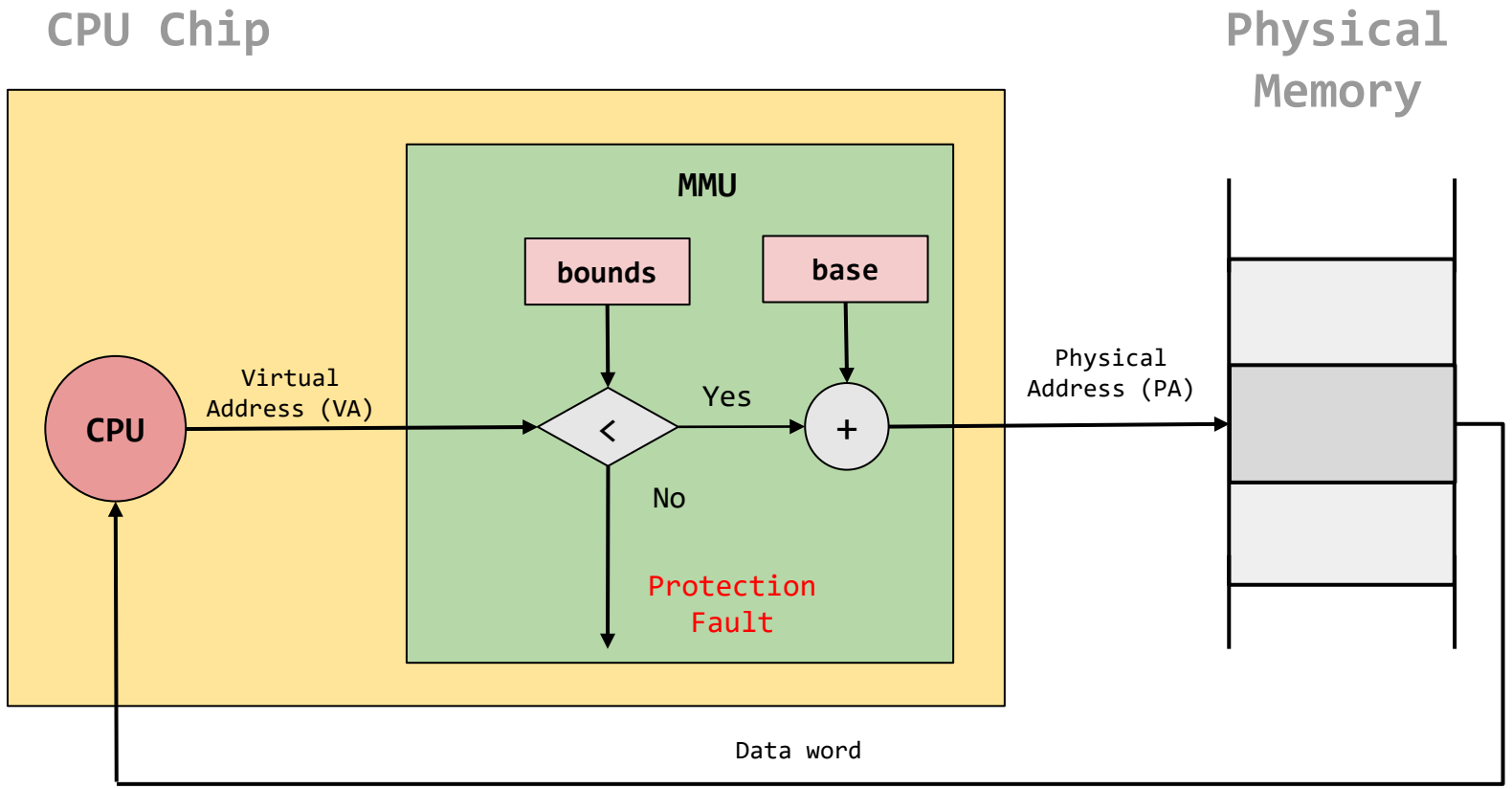
Facultad de Ingeniería

Ingeniería de Sistemas

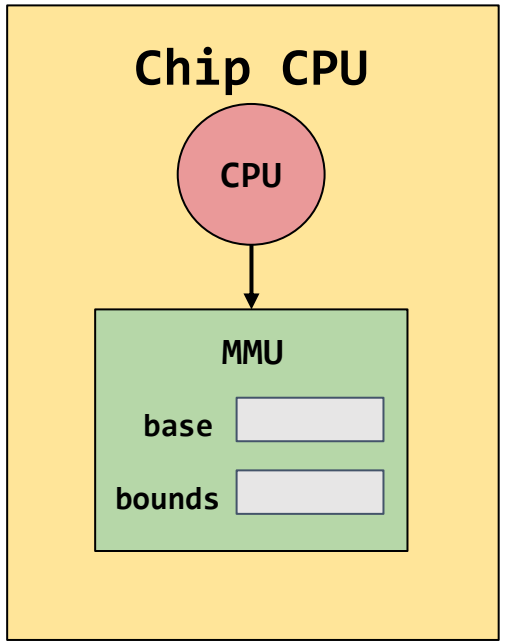
01/10/2026

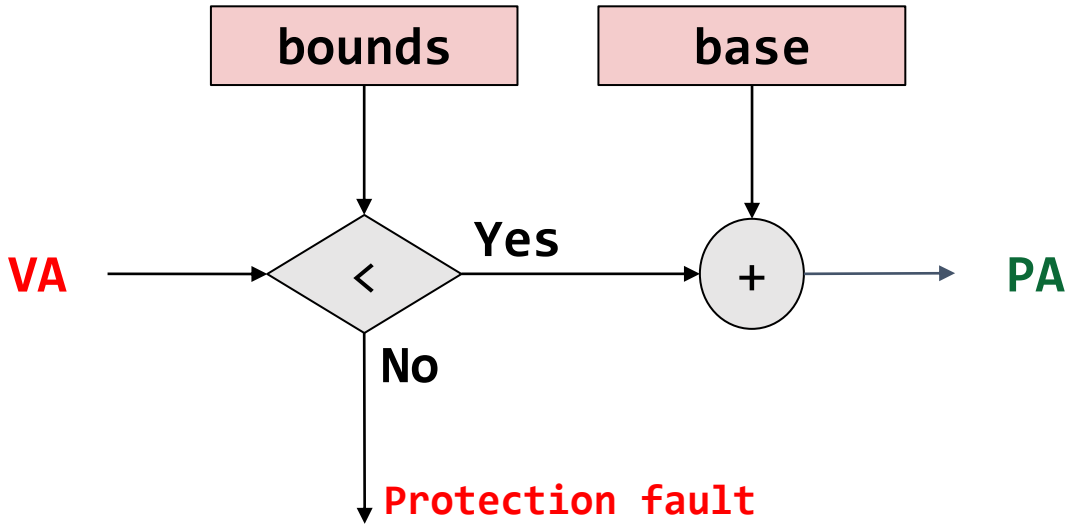
Dynamic Relocation (base & bounds)





Dynamic Relocation (base & bounds)

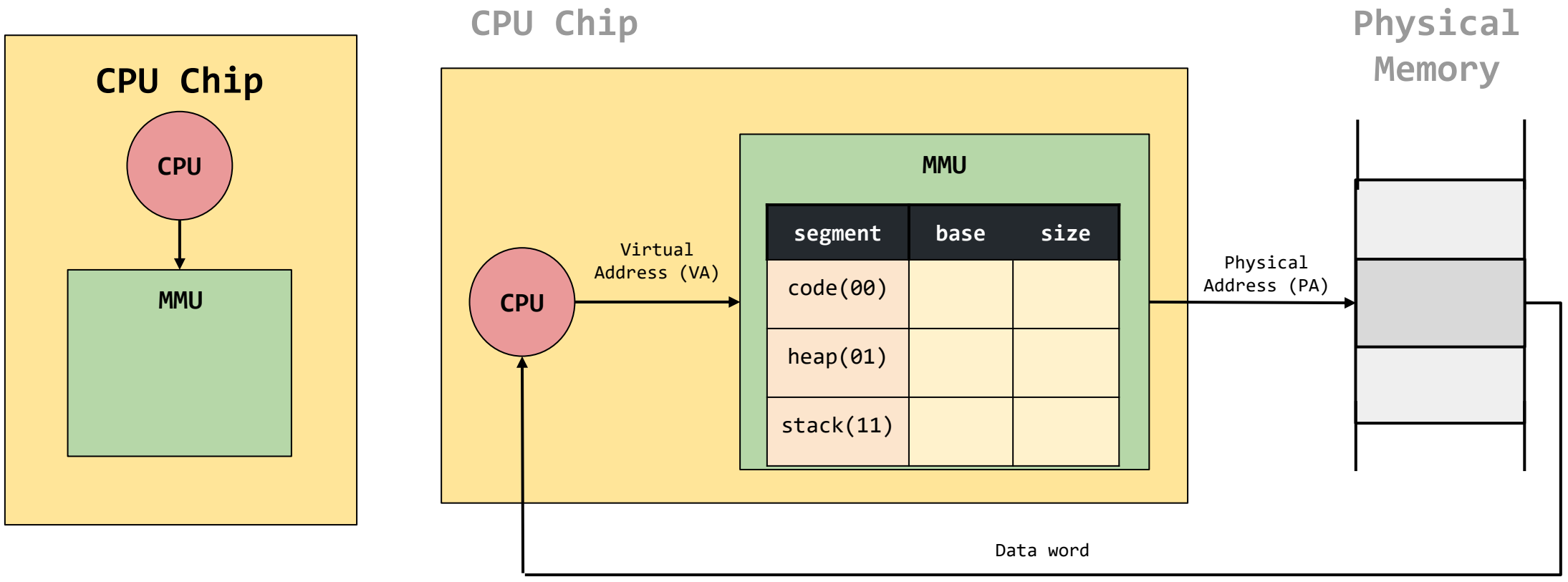




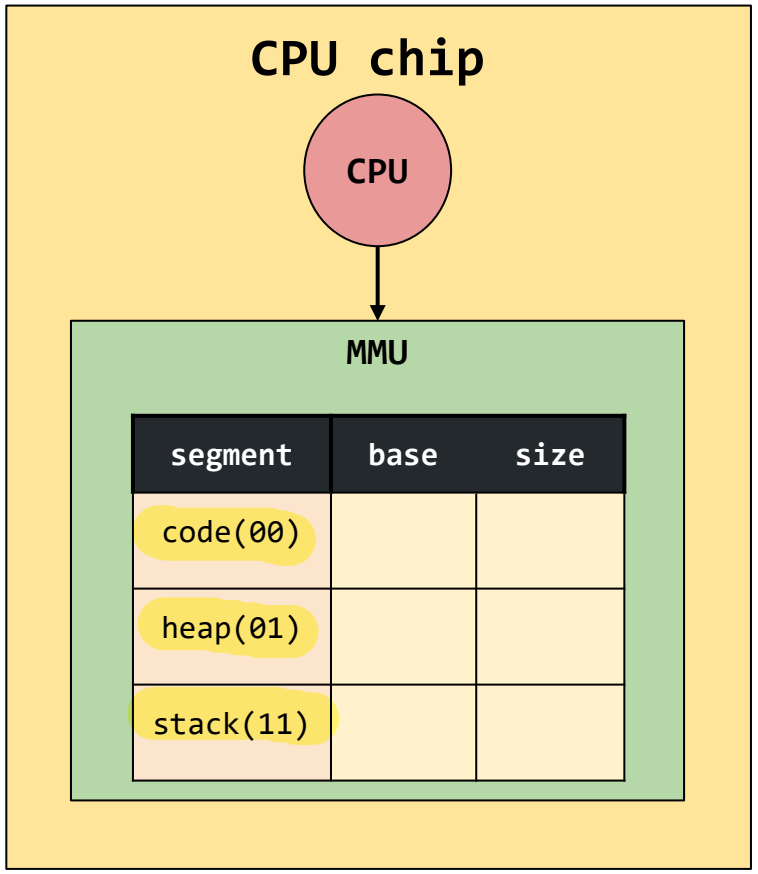

```
PA = VA + base
```

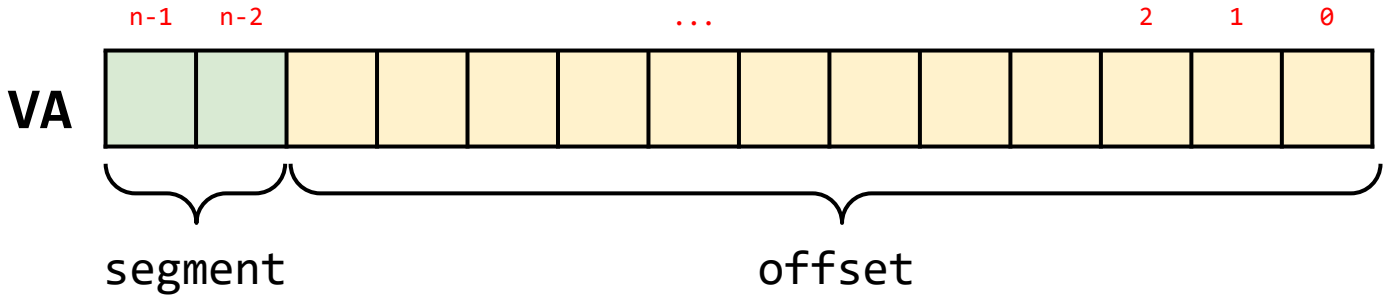
```
0 <= VA < bounds
```

Segmentación



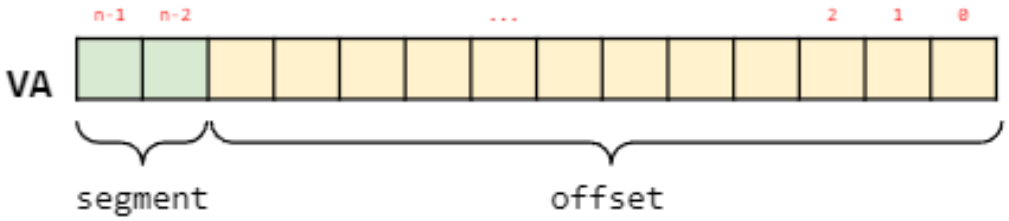
Segmentación



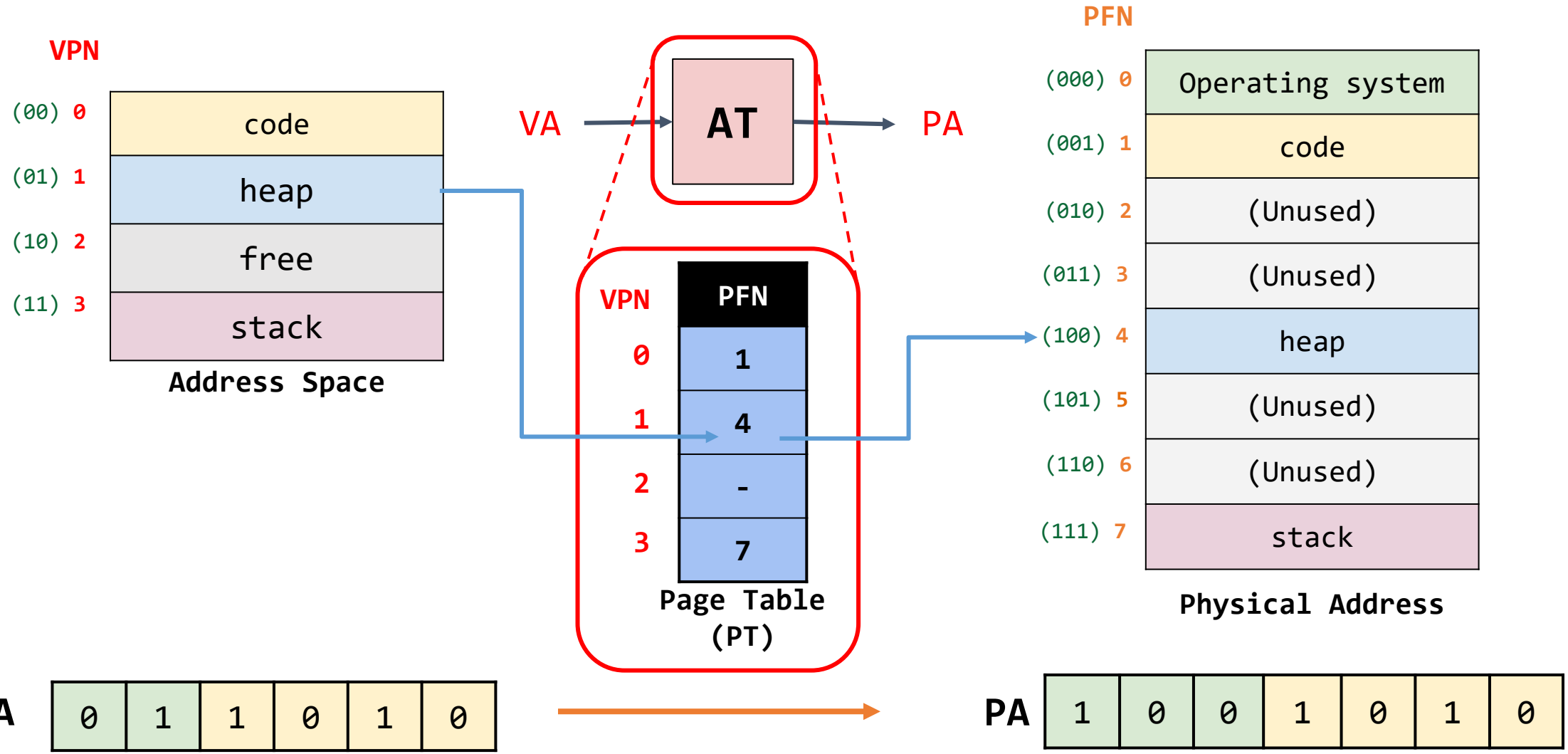


segment	base	size	Grows Positive?	Protection
code(00)				Read-Execute
heap(01)				Read-Write
stack(11)				Read-Write

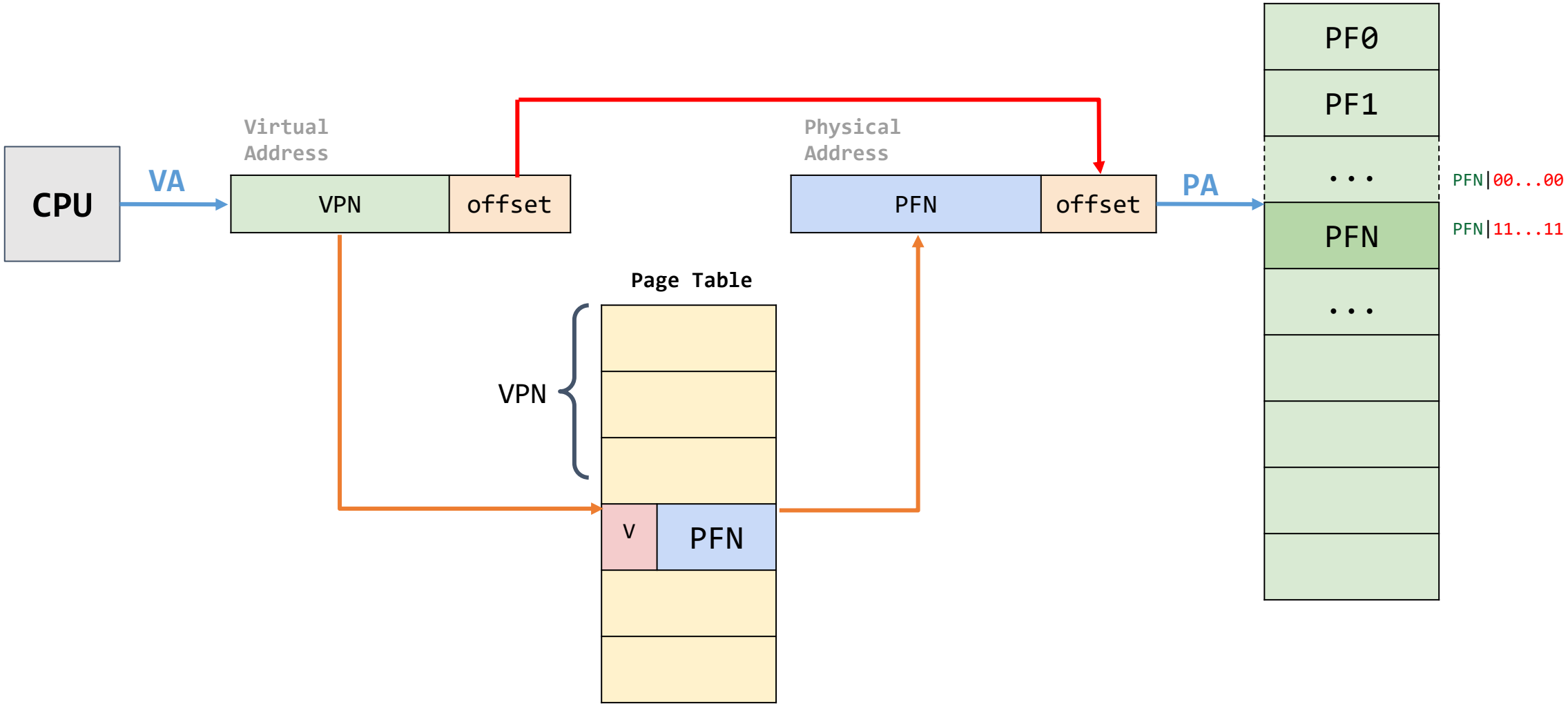
Segmentación

segment	**offset**	**Physical Address**
code	`offset = VA`	`PA = offset + base(code)`
heap	`offset = VA - VA(heap)`	`PA = offset + base(heap)`
stack	 `offset(stack) = offset - offset(max)`	`PA = offset + base(stack)`

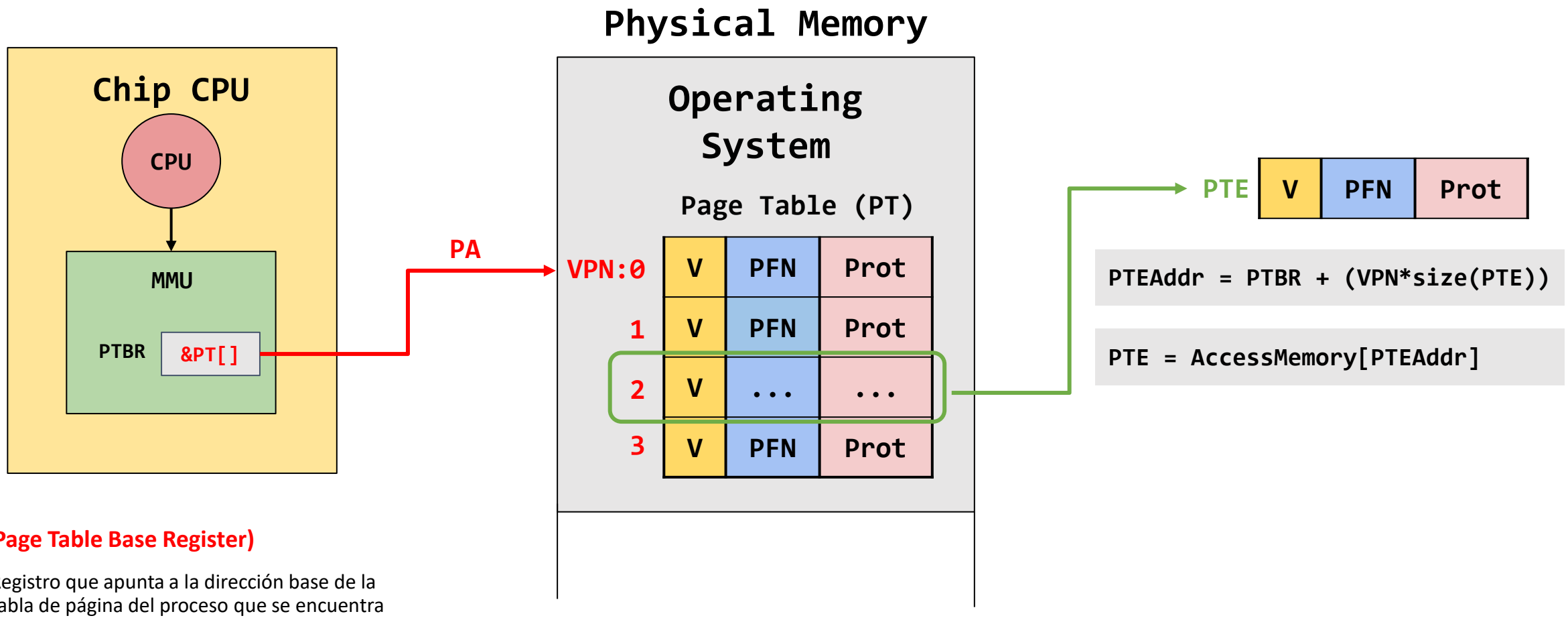
Paginación


VPN
(00) 0
(01) 1
(10) 2
(11) 3
code
heap
free
stack
Address Space
VA
AT
PA
VPN
PFN
0
1
1
4
2
-
3
7
Page Table
(PT)
PFN
(000) 0
(001) 1
(010) 2
(011) 3
(100) 4
(101) 5
(110) 6
(111) 7
Operating system
code
(Unused)
(Unused)
heap
(Unused)
(Unused)
stack
Physical Address
VA
0 1 1 0 1 0
PA
1 0 0 1 0 1 0

Paginación


CPU
VA
Virtual Address
VPN
offset
Physical Address
PFN
offset
PA
PF0
PF1
...
PFN
...
PFN|00...00
PFN|11...11
Page Table
VPN
V
PFN

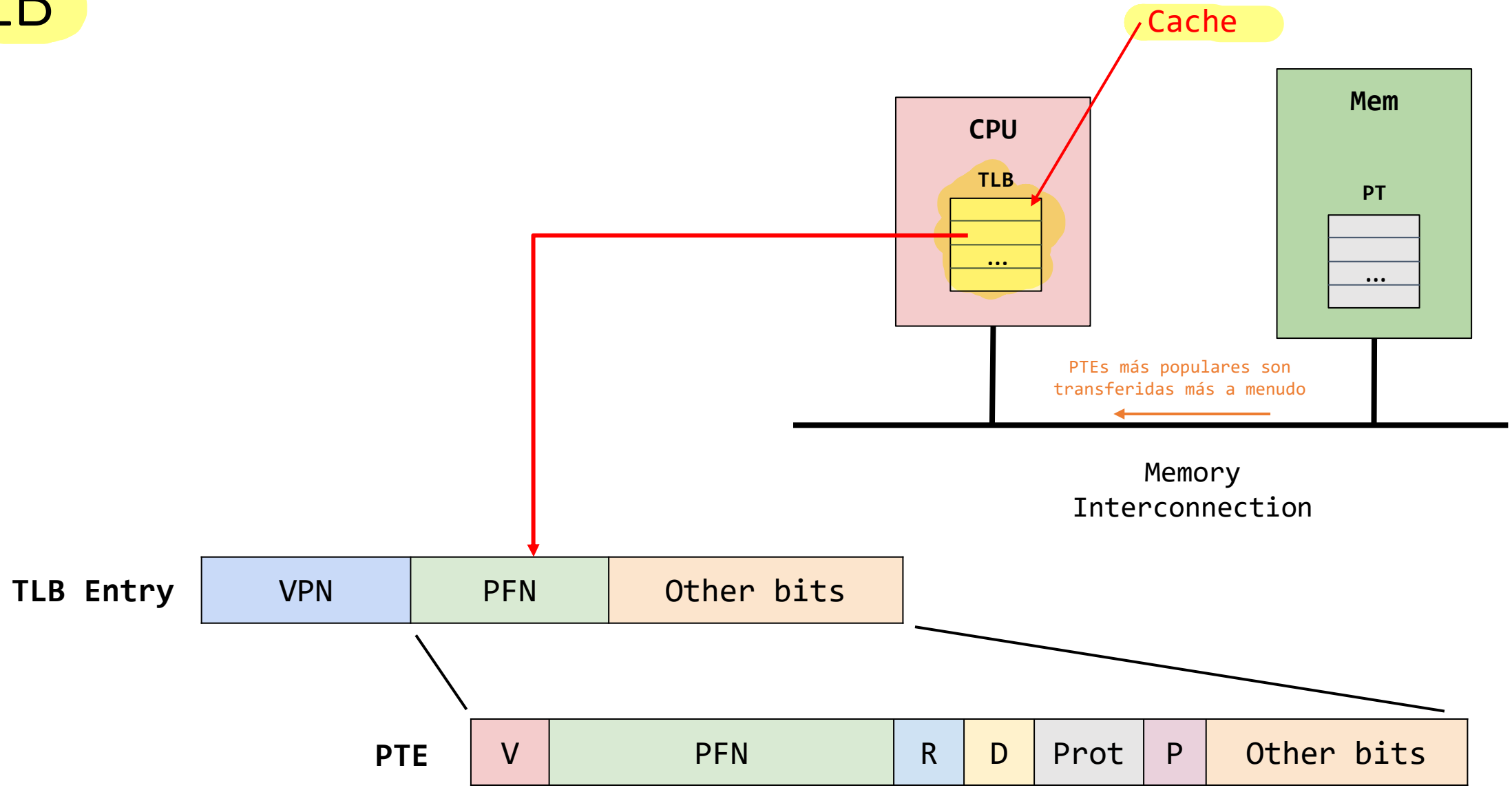
Paginación



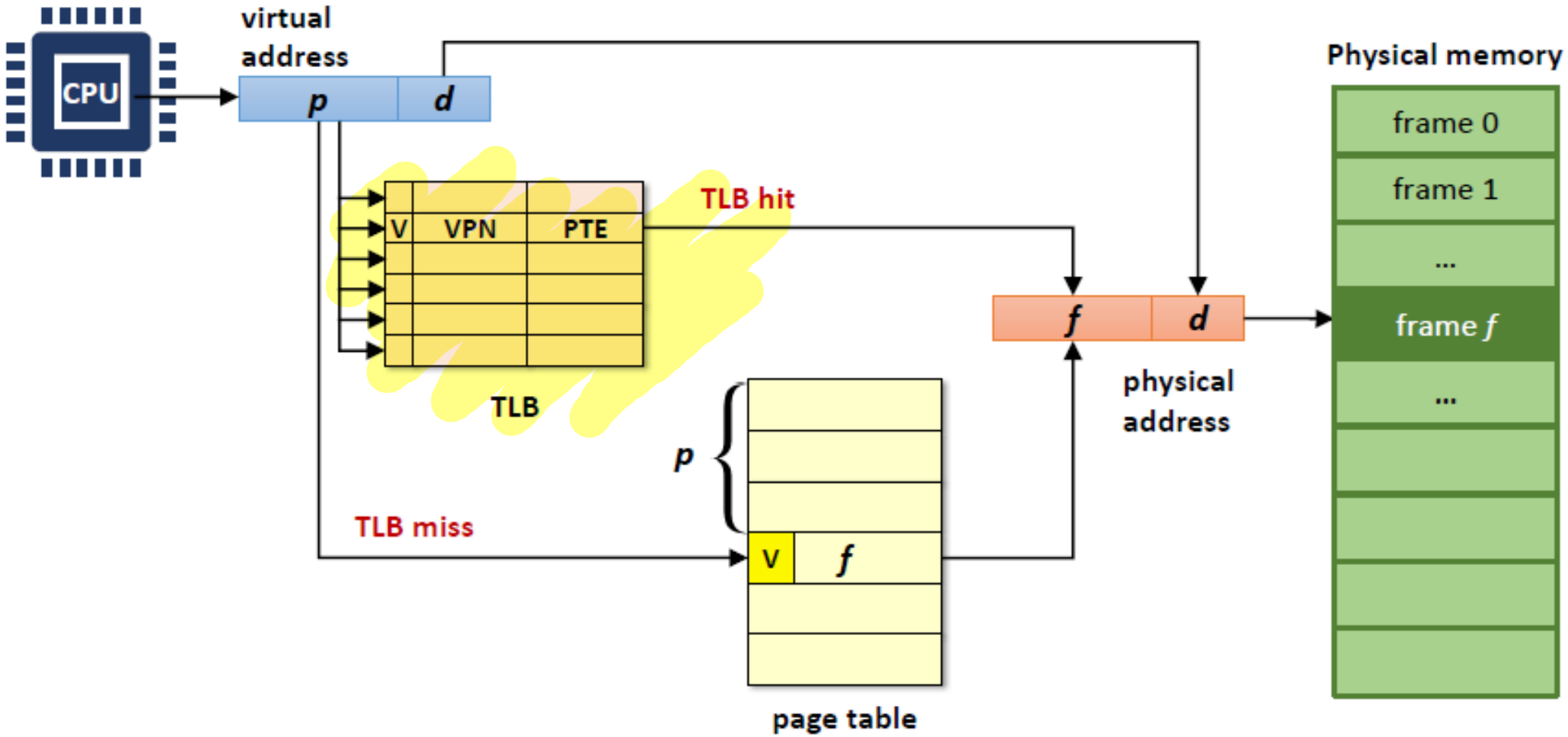
PTBR (Page Table Base Register)

- Registro que apunta a la dirección base de la tabla de página del proceso que se encuentra actualmente en ejecución.
- Cada proceso tiene su propio PTBR.

TLB


Cache
CPU
TLB
...
Mem
PT
...
PTEs más populares son
transferidas más a menudo
Memory
Interconnection
TLB Entry
VPN
PFN
Other bits
PTE
V
PFN
R
D
Prot
P
Other bits

TLB


CPU
virtual address
p
d
V
VPN
PTE
TLB hit
TLB
TLB miss
p
V
f
page table
f
d
physical address
Physical memory
frame 0
frame 1
...
frame f
...

Problemas debido a los métodos de traducción de direcciones

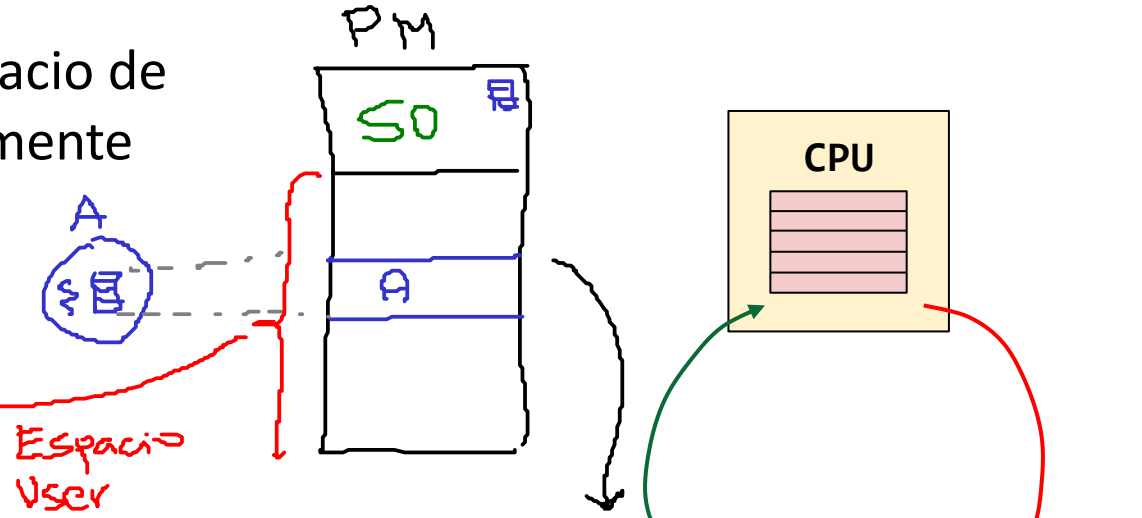
Método	**Problemas**
Dynamic Relocation (Base & Bounds)	Fragmentación interna
Segmentation (Base & Bounds generalizado)	Fragmentación externa
Paginación	Enorme gasto de memoria debido al espacio ocupado en memoria por cada una de las tablas de pagina asociadas al workload → Overhead de memoria. Doble acceso a memoria → Ralentización
TLB	Gasto de Memoria → Overhead de memoria Poco optima cuando no hay localidad → TLB miss altas.

Swapping - Contextualización

Suposiciones hechas hasta el momento

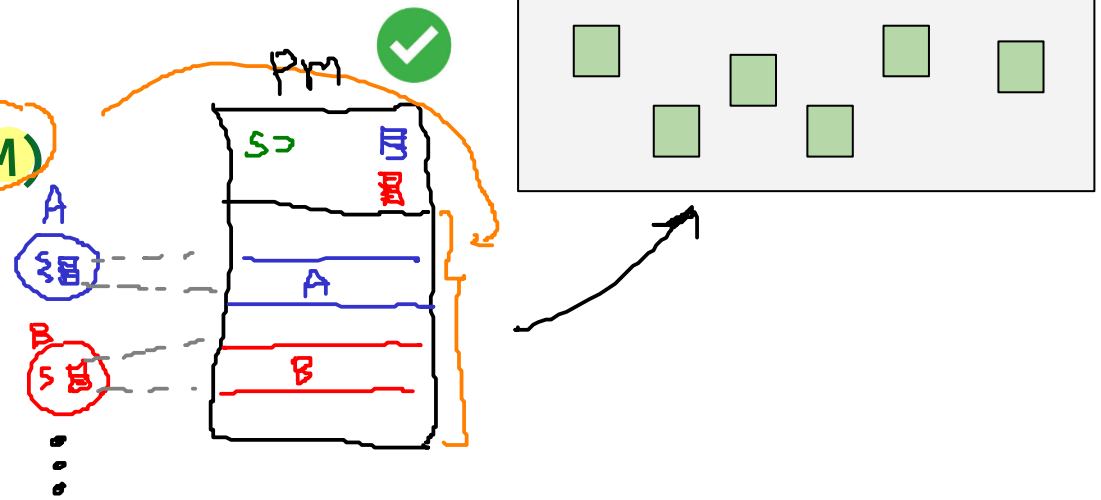
- Anteriormente habíamos asumido que el espacio de direcciones (Address Space) era lo suficientemente pequeño, para caber en la memoria física.

`size(AS) < size(PM)`



- Incluso, los espacios de direcciones de todos los procesos en ejecución caben en memoria física.

`size(sum(AS[Pi])) < size(PM)`



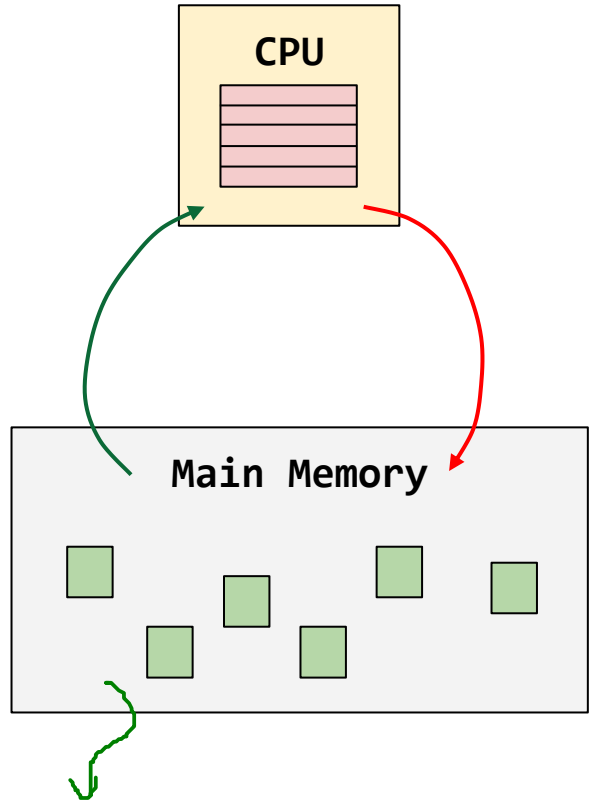
Swapping - Contextualización

Relajando las situaciones anteriores

- ¿Es posible que se den casos en los cuales el SO no tenga suficiente memoria fisica (PM) para soportar todos los procesos? - Si
- ¿Cuando se puede dar lo anteriormente planteado?
  - Cuando se tiene un único proceso con un espacio de direcciones muy grande.

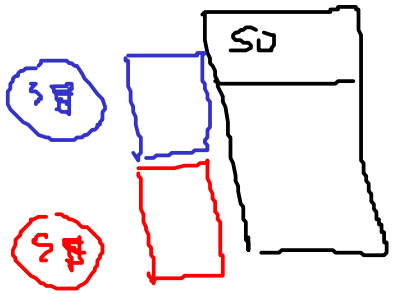
`size(AS) > size(PM)`





  - Cuando el espacio de direcciones de todos los procesos combinados supera la cantidad de memoria física disponible.

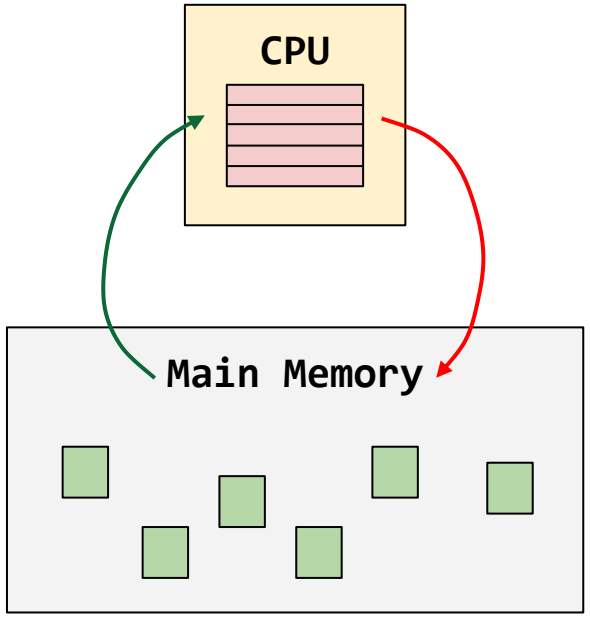
`size(sum(AS[Pi])) > size(PM)`



Swapping - Contextualización

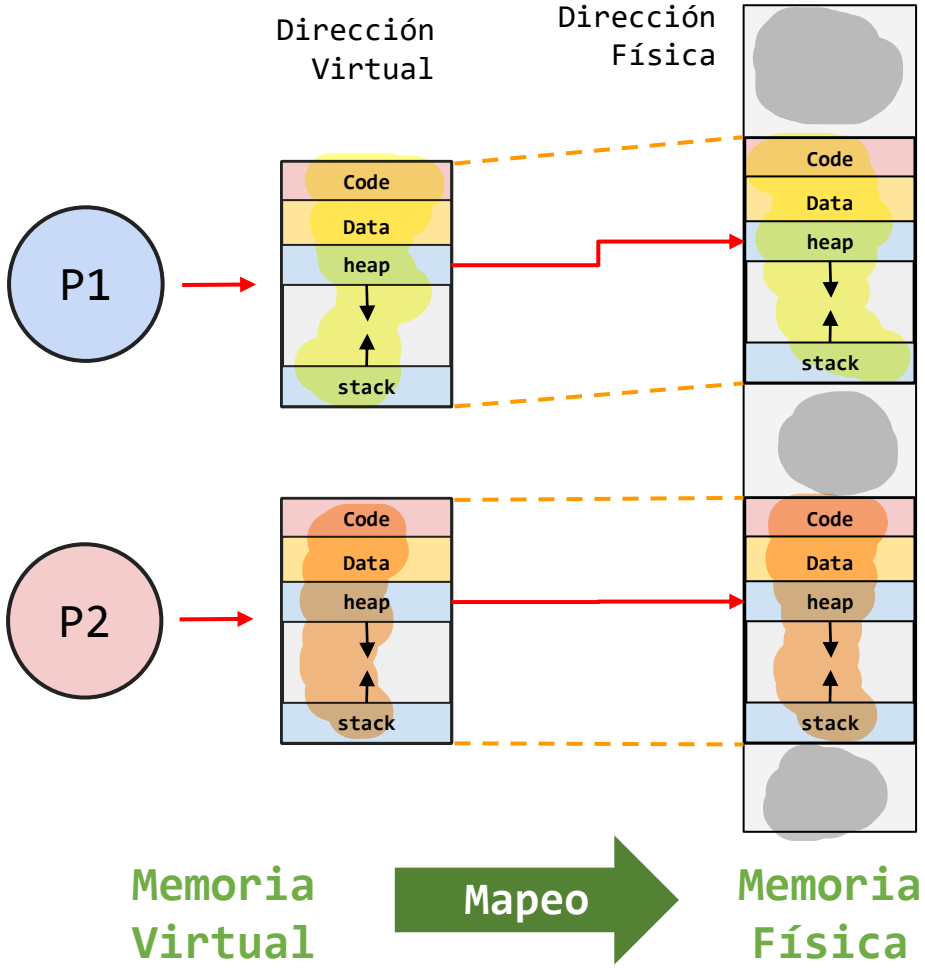
En resumen

1. El espacio de direcciones de un proceso es pequeño y cabe en memoria física.
2. El espacio de direcciones debido al **workload** (todos los procesos en ejecución) cabe en memoria.



No tenemos suficiente Mem. Fisica

Que el SO en este caso



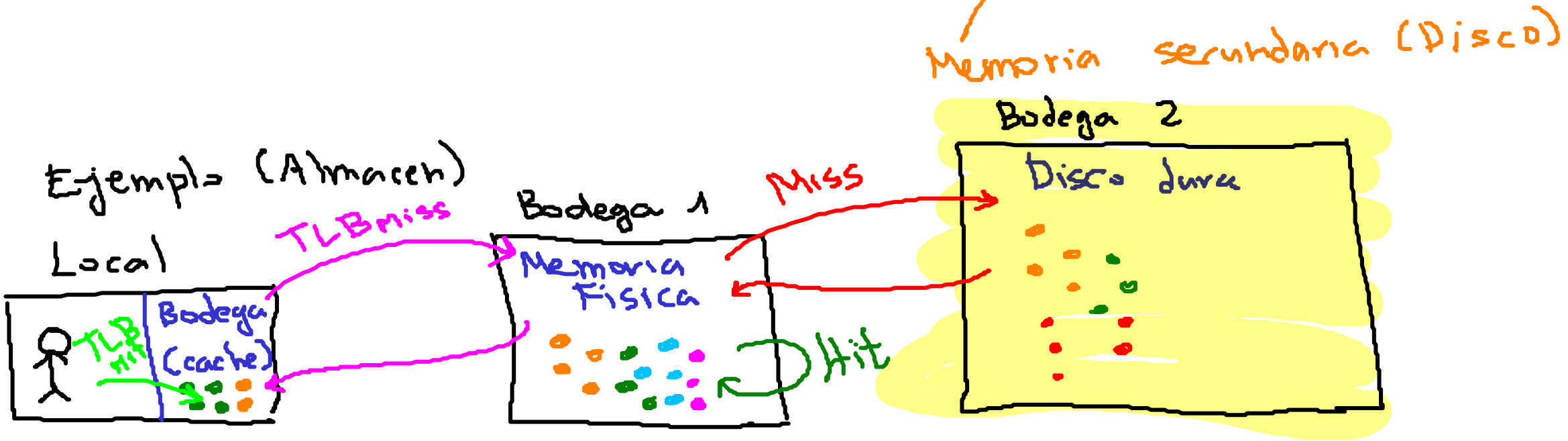


Swapping - Contextualización

Pregunta clave

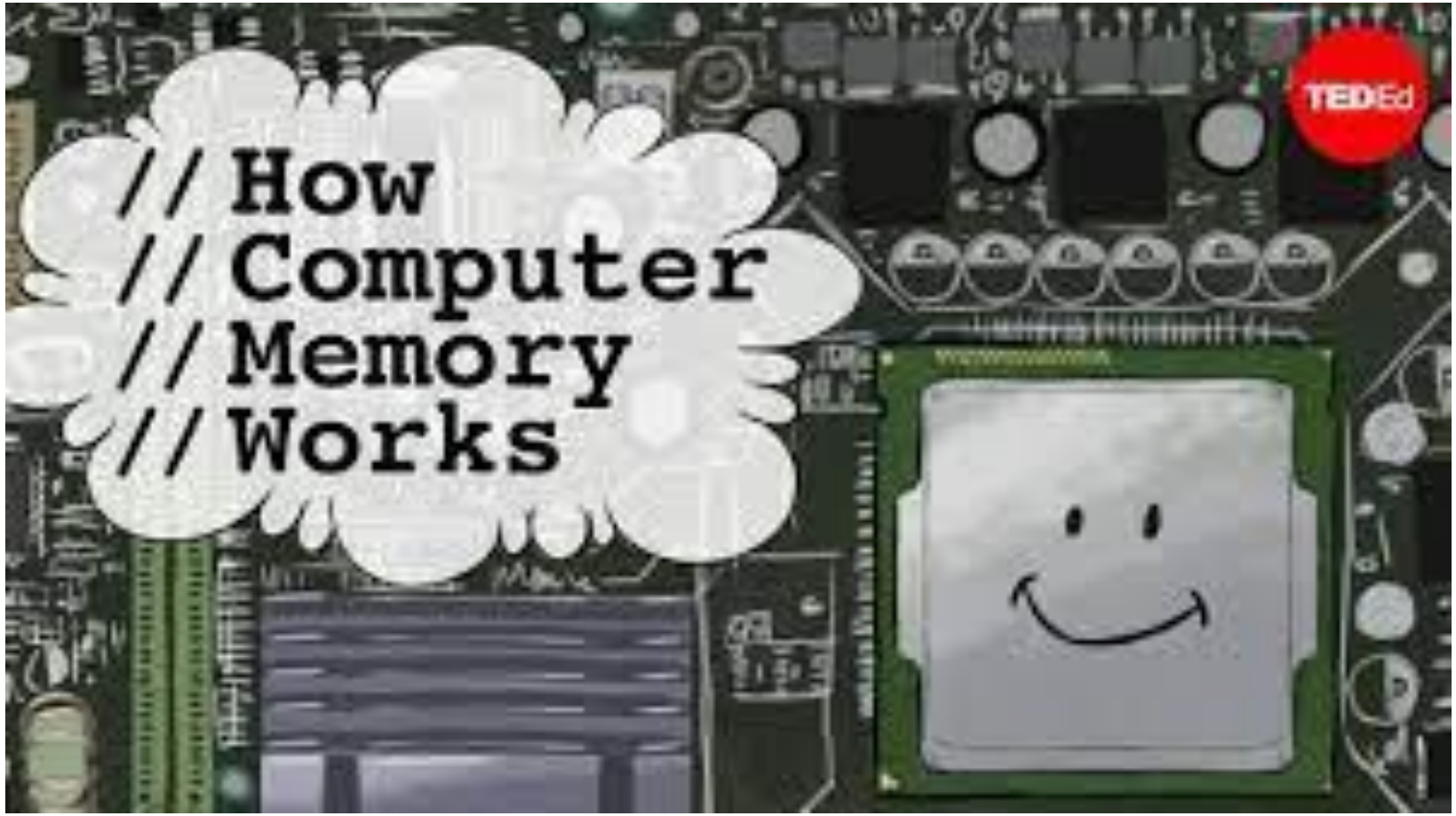
¿Cómo ir más allá de la memoria física?

¿Cómo puede el sistema operativo usar un dispositivo más grande y lento para proporcionar de forma transparente la ilusión de un gran espacio de direcciones virtuales?



Jerarquía de memoria

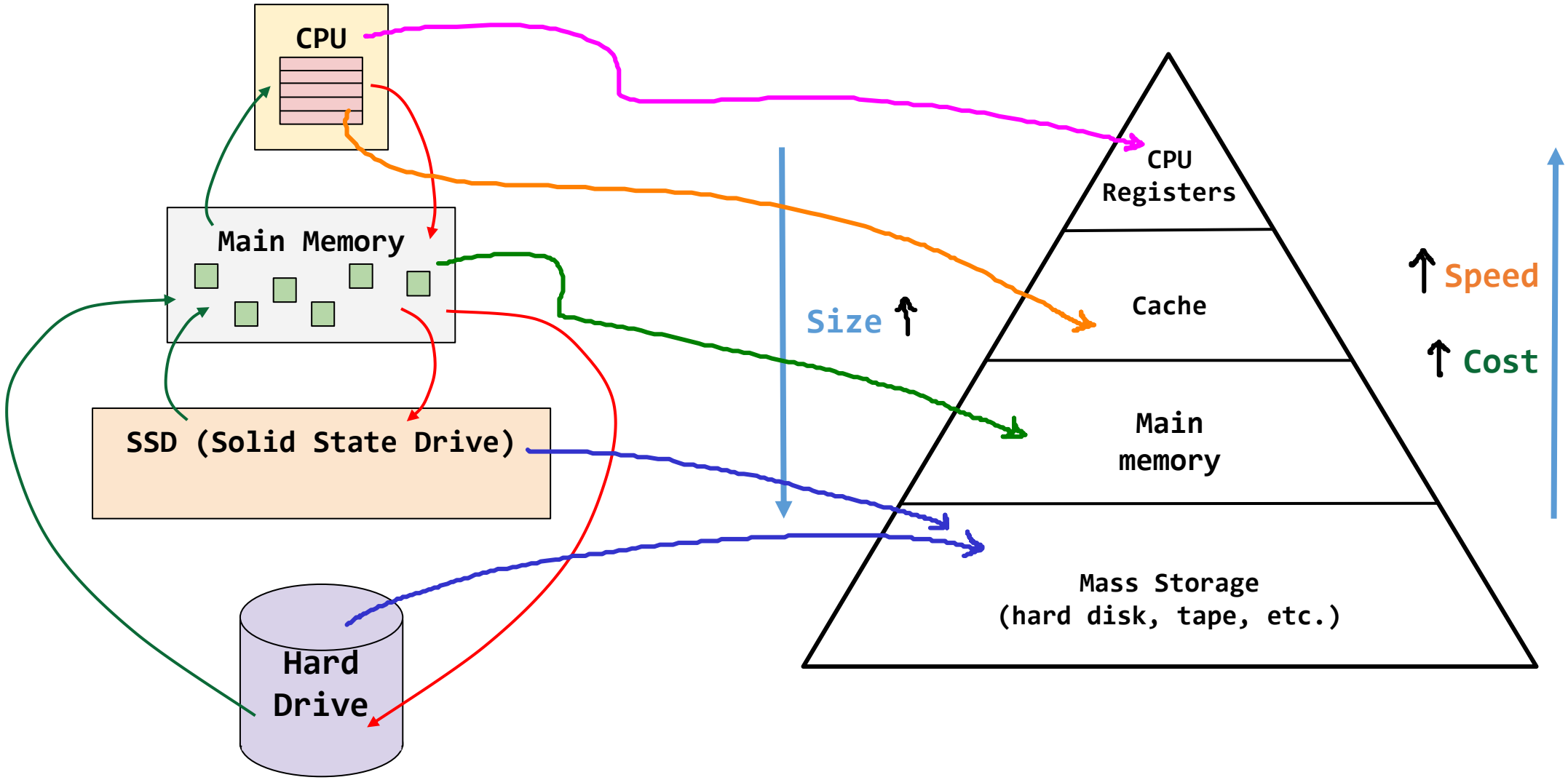
Jerarquía de memoria en un sistema moderno



How computer memory works - Kanawat Senanan (link)

Jerarquía de memoria

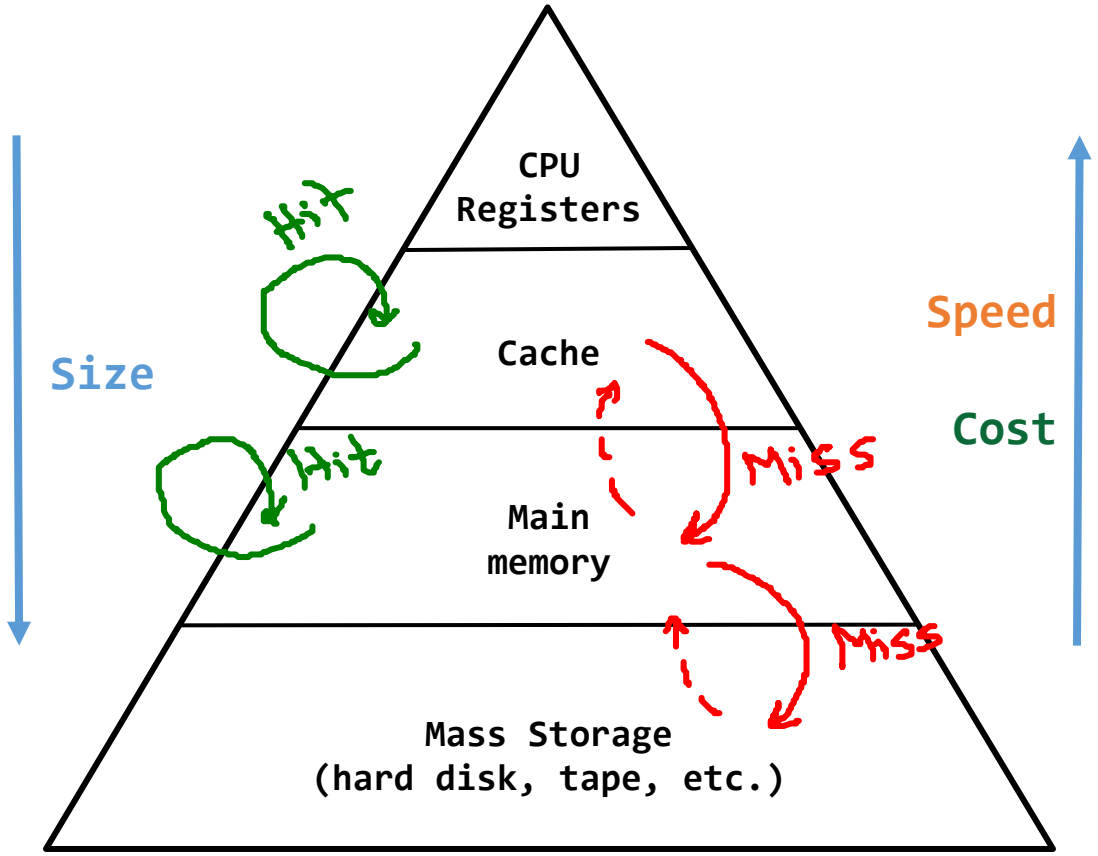
Jerarquía de memoria en un sistema moderno



Jerarquía de memoria

Jerarquía de memoria en un sistema moderno

- Cada capa actúa como un **espacio de almacenamiento de respaldo (backing store)** de la capa superior



Jerarquía de memoria

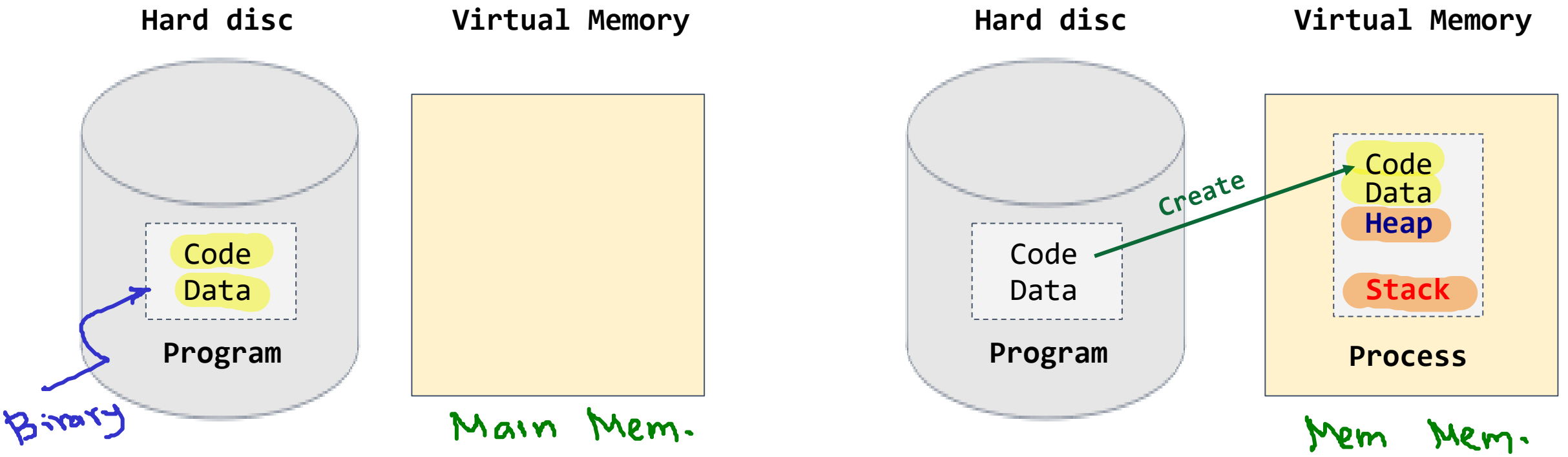
Procesos y memoria

Ejecución de un programa sencillo

1. Estado inicial

2. Ejecución del programa

   Se crea un proceso y se reserva memoria para este.

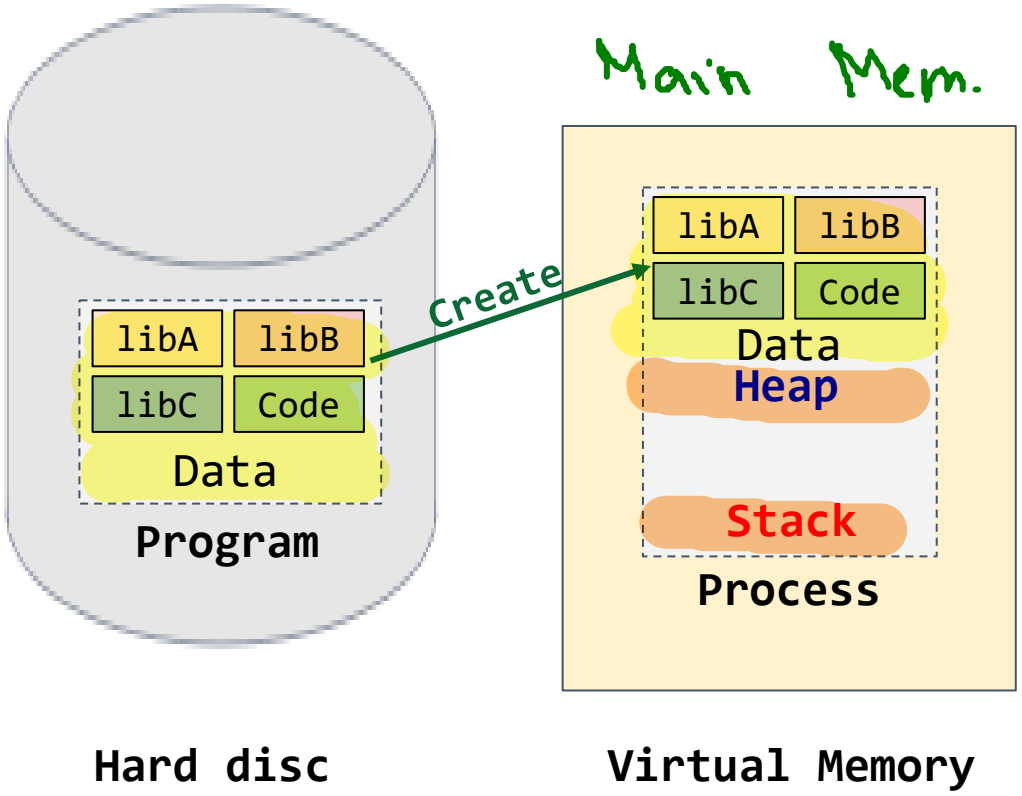


Jerarquía de memoria

Procesos y memoria

Programa más realista: Suele hacer uso de muchas librerías algunas de las cuales raramente suelen ser usadas.

Uso de librerias estaticas



Escenario

- El programa usa varias librerías que son referenciadas en memoria virtual.
- De acuerdo a lo visto hasta ahora, las páginas virtuales tienen que estar mapeadas en memoria física a través de la **tabla de página (PT)**.

Problema: Gasto de memoria

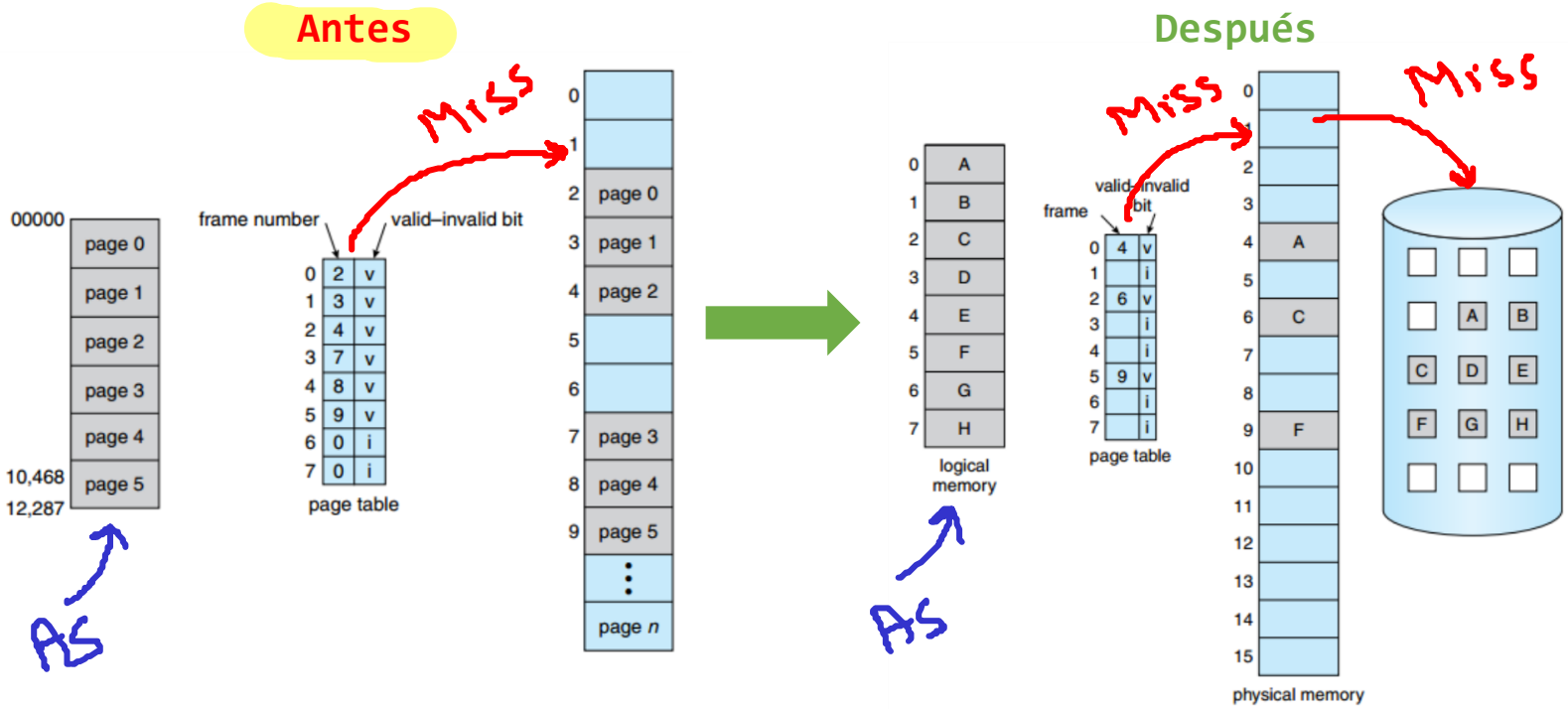
Jerarquía de memoria

Procesos y memoria

Escenario

¿Cómo evitar el desperdicio de páginas físicas (**frames**) empleados para respaldar páginas virtuales (**pages**) poco usadas?

- Se requiere un espacio (usualmente en el disco duro) en la jerarquía de memoria.
- Enviar a otro nivel porciones de espacio de direccionamiento que **no son muy demandadas**.



Swapping - Introducción

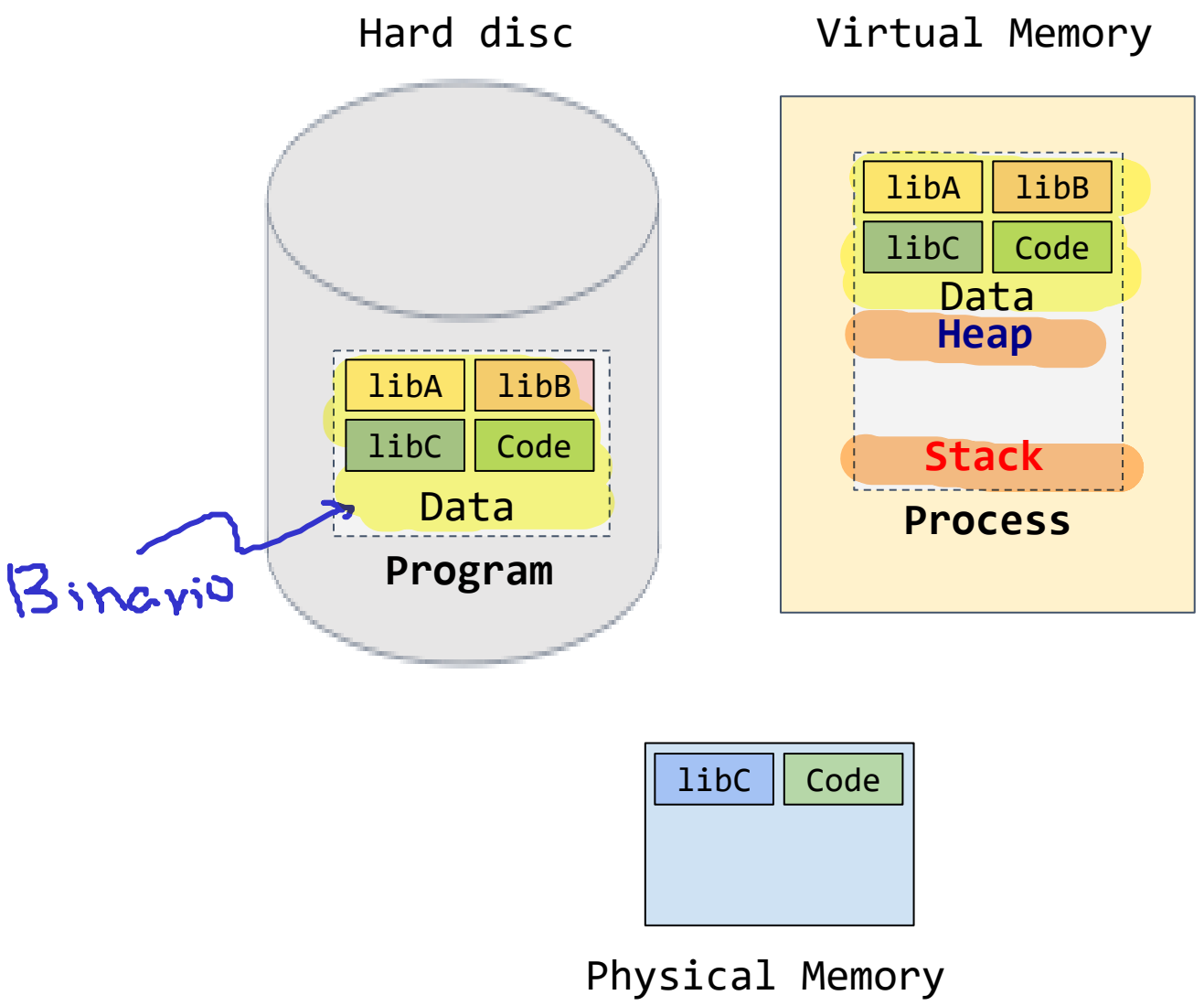
¿Cómo ir más allá de la memoria física?
¿Cómo puede el sistema operativo usar un dispositivo más grande y lento para proporcionar de forma transparente la ilusión de un gran espacio de direcciones virtuales?

Mejora: Swapping

Más allá de la memoria física

¿Cómo funciona?

1. **Ejecución del programa**
   Se reserva la memoria virtual del proceso creado.

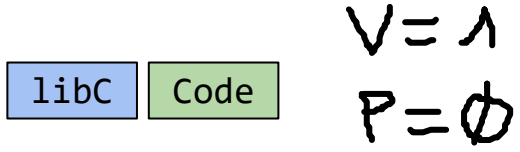


Más allá de la memoria física

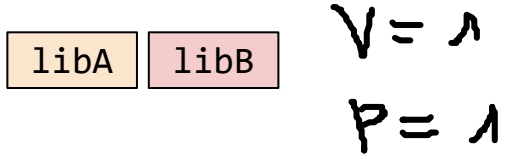
¿Cómo funciona?

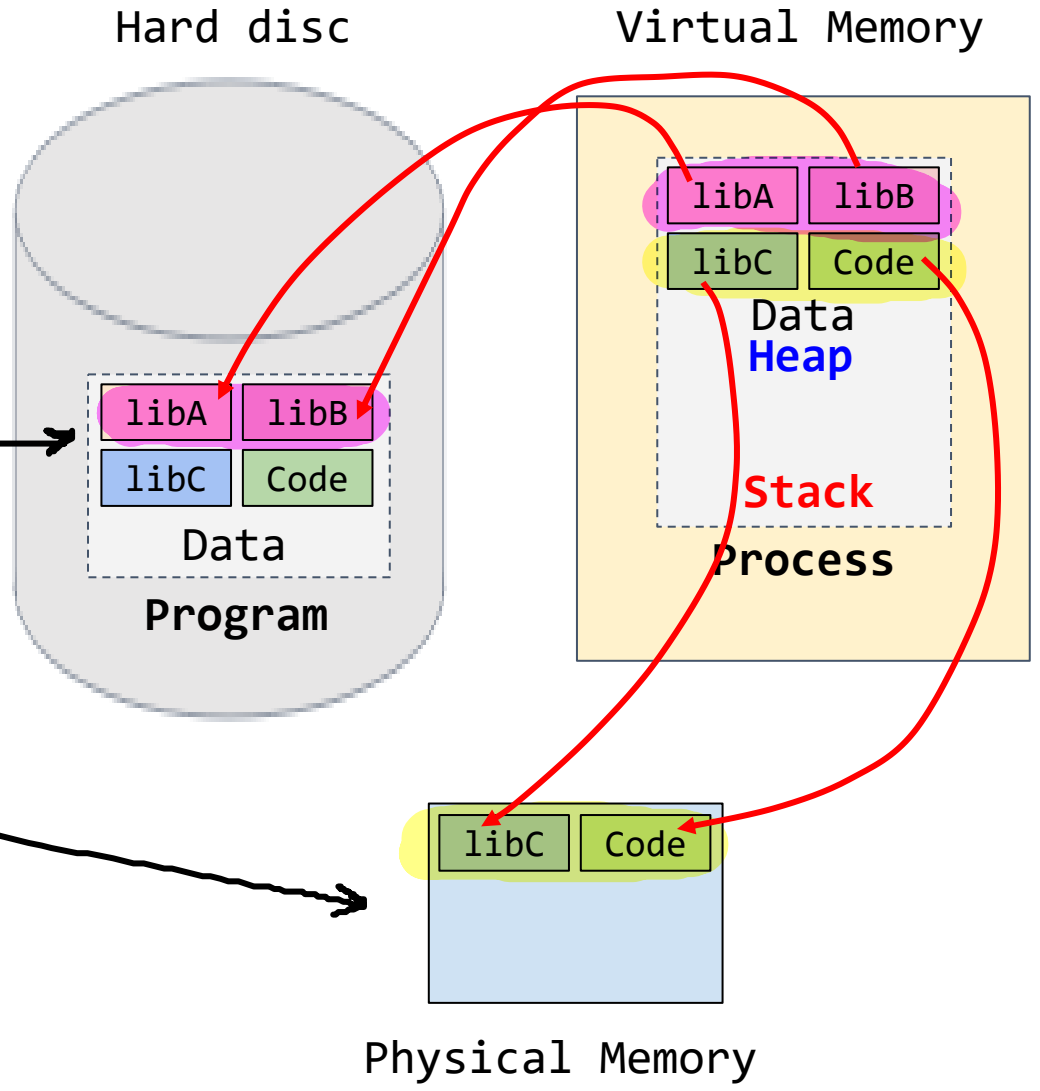
2. **Se crean las referencias**

- Memoria virtual → Memoria física



- Memoria virtual → Disco duro



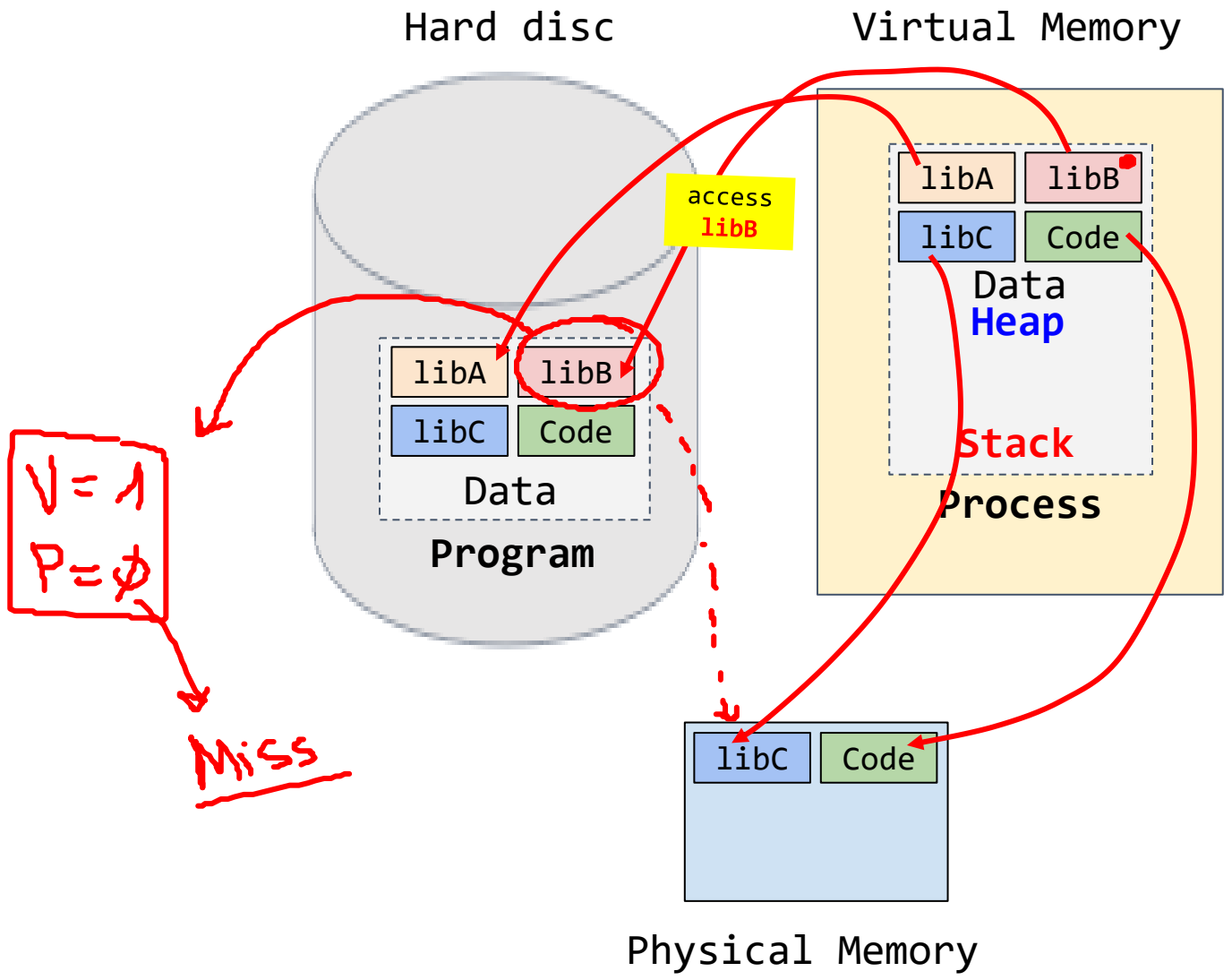


Más allá de la memoria física

¿Cómo funciona?

3. Se accede a la librería B

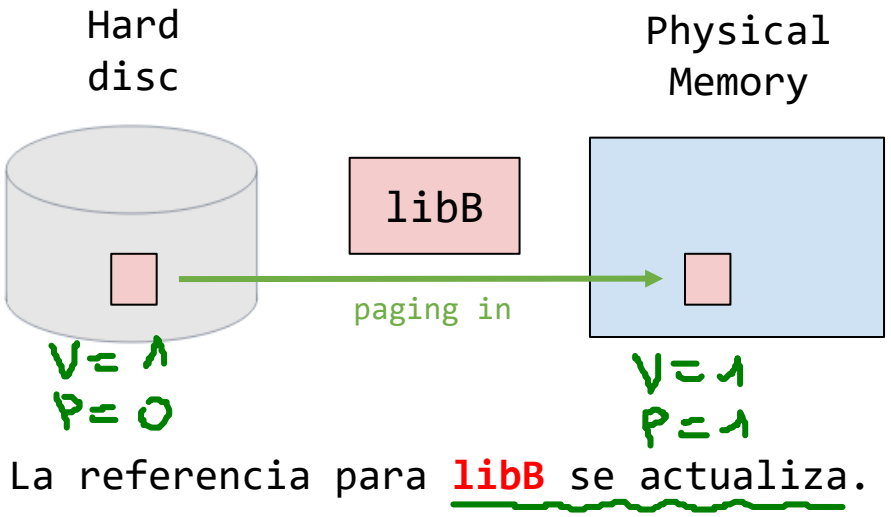
libB

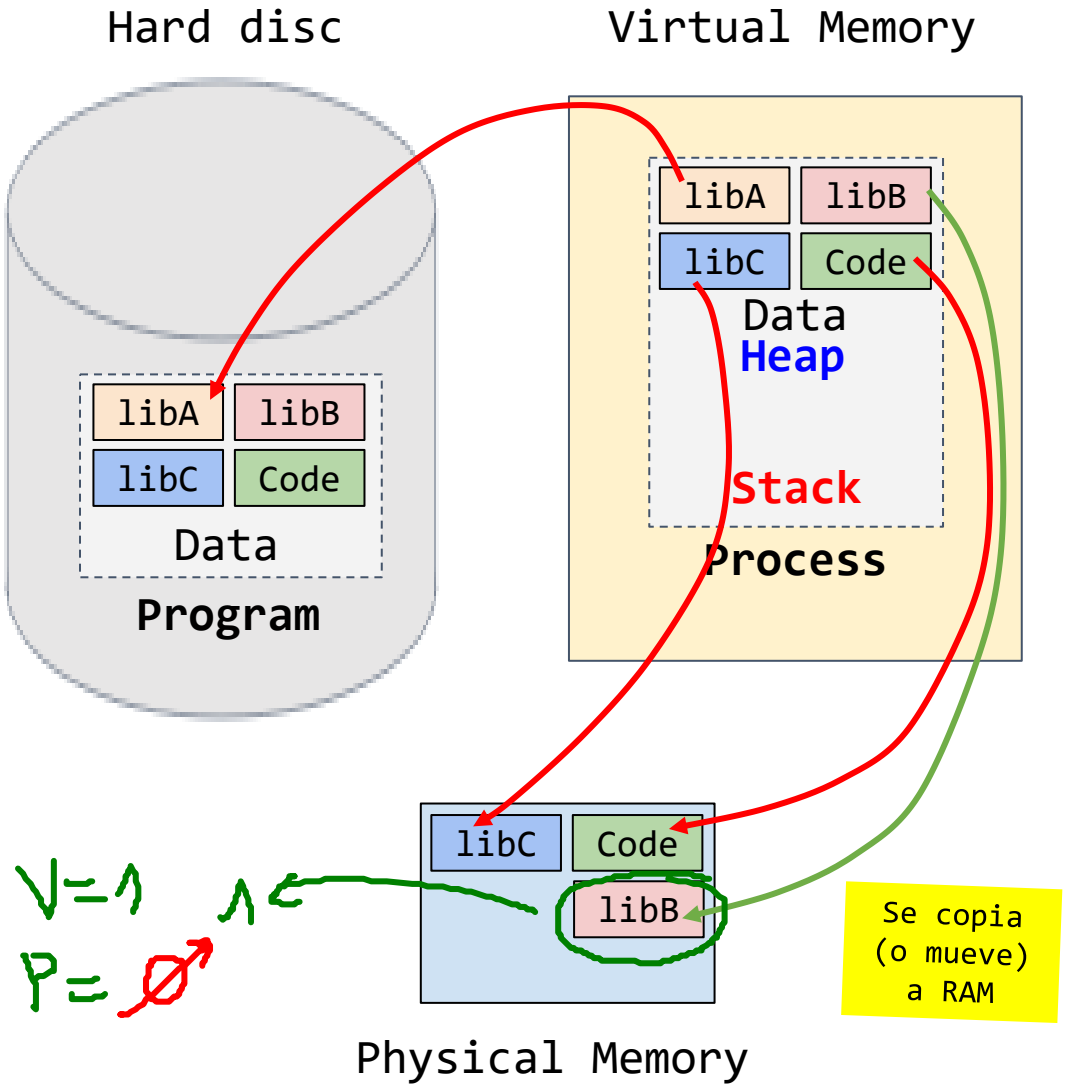


Más allá de la memoria física

¿Cómo funciona?

4. **Se copia (o mueve) la página a la memoria principal**

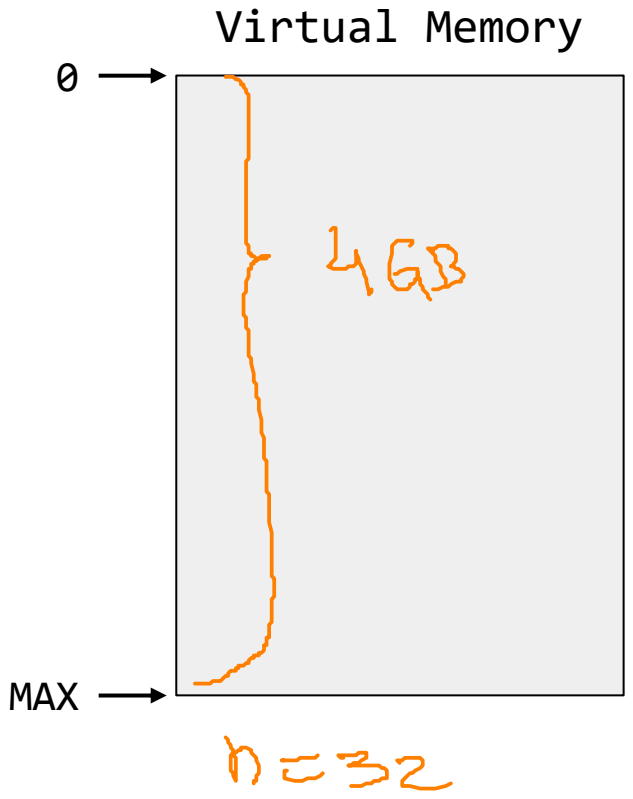




Más allá de la memoria física

Problema de los grandes espacios de direccionamiento

Escenario: Veamos un caso real:



Bits de Mem. virtual	Tamaño de la Mem. virtual
32	4*(2^30) = 4 GB
64	16*(2^60) = 16 EB **(Valor muy grande)**

$n=32 \rightarrow [0, 2^{32}-1]$

$2^{32} = (2^2)(2^{30}) = 4GB$

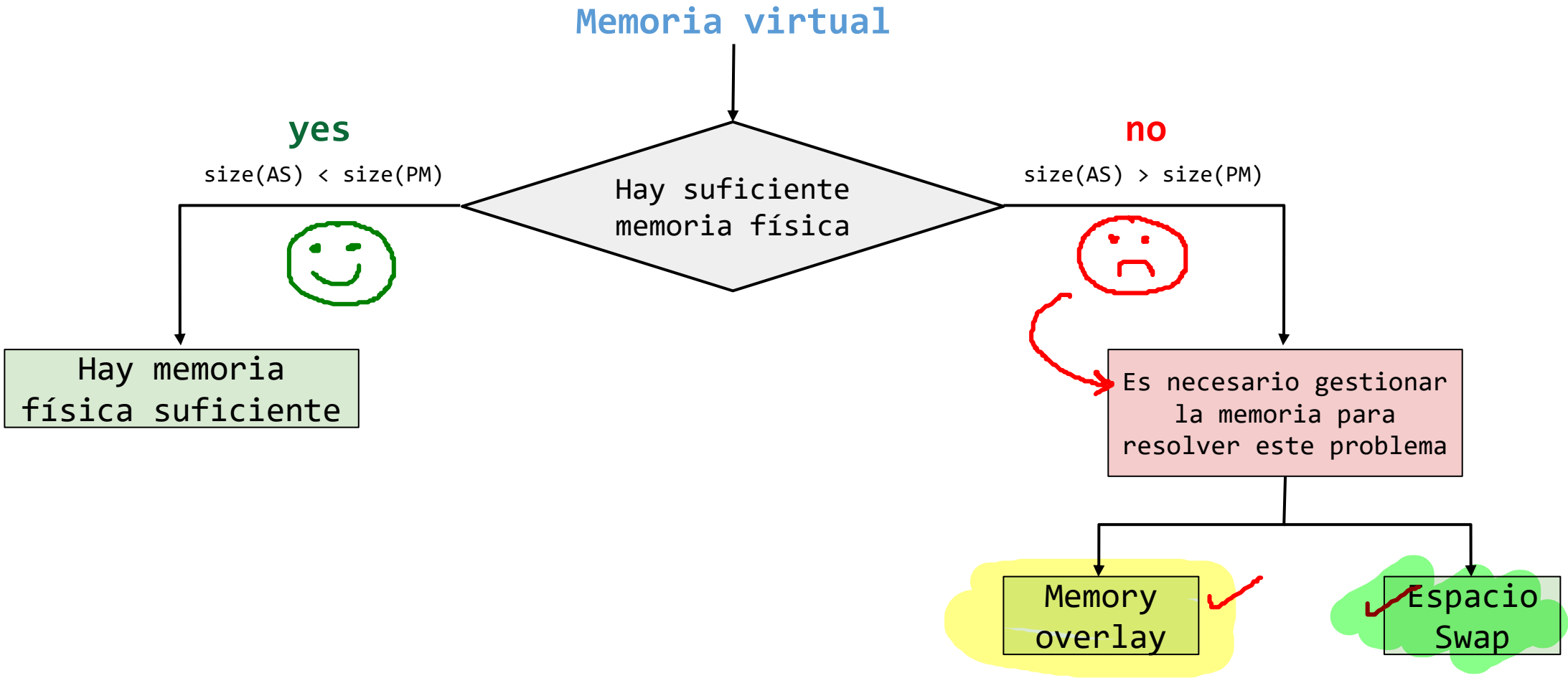
$2^{64} = 2^4(2^{60}) = 2^4 EB$

Conclusiones:

- El espacio de memoria virtual es muy grande, pudiendo incluso superar el tamaño de la memoria física (real).
- Bajo estas condiciones, puede que ni siquiera quepa un proceso (y mucho menos varios).

Más allá de la memoria física

Problema de los grandes espacios de direccionamiento



Más allá de la memoria física

Memory overlay (Manual)

Escenario:

El programa tiene mayor tamaño (memoria virtual) que la memoria física disponible lo cual hace que el programa no pueda ser cargado.

`size(AS) > size(PM)`

Posibles soluciones:

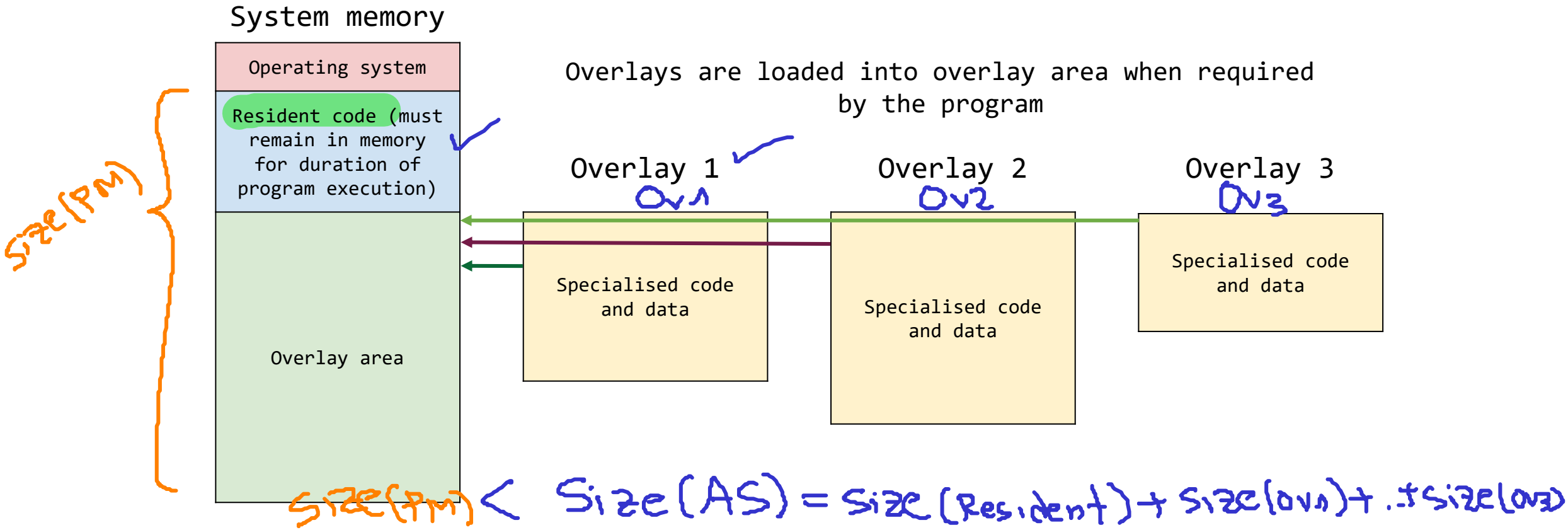
1. **Aumentar la memoria** (Costoso)
2. **Memory overlay - superposición de memoria** (Aproximación usada en los sistemas operativos viejos)

Más allá de la memoria física

Memory overlay (Manual)

El programador divide el programa en un número de secciones lógicas:

- **Código residente**: Sección lógica que permanece siempre memoria.
- **Overlay area**: Secciones del programa que solo se cargan cuando son requeridas.

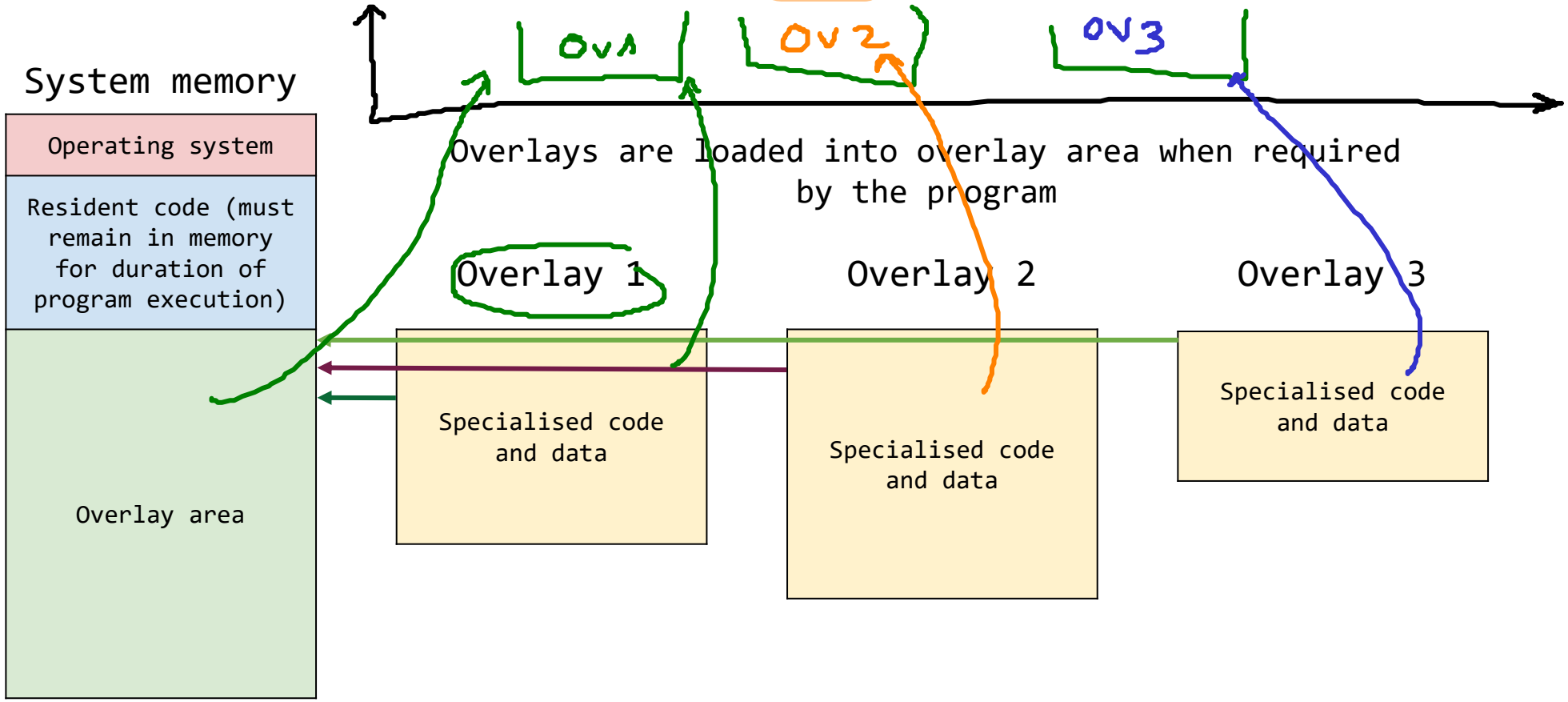


Más allá de la memoria física

Memory overlay (Manual)

El uso de memoria corre por cuenta del programador y no del sistema operativo:

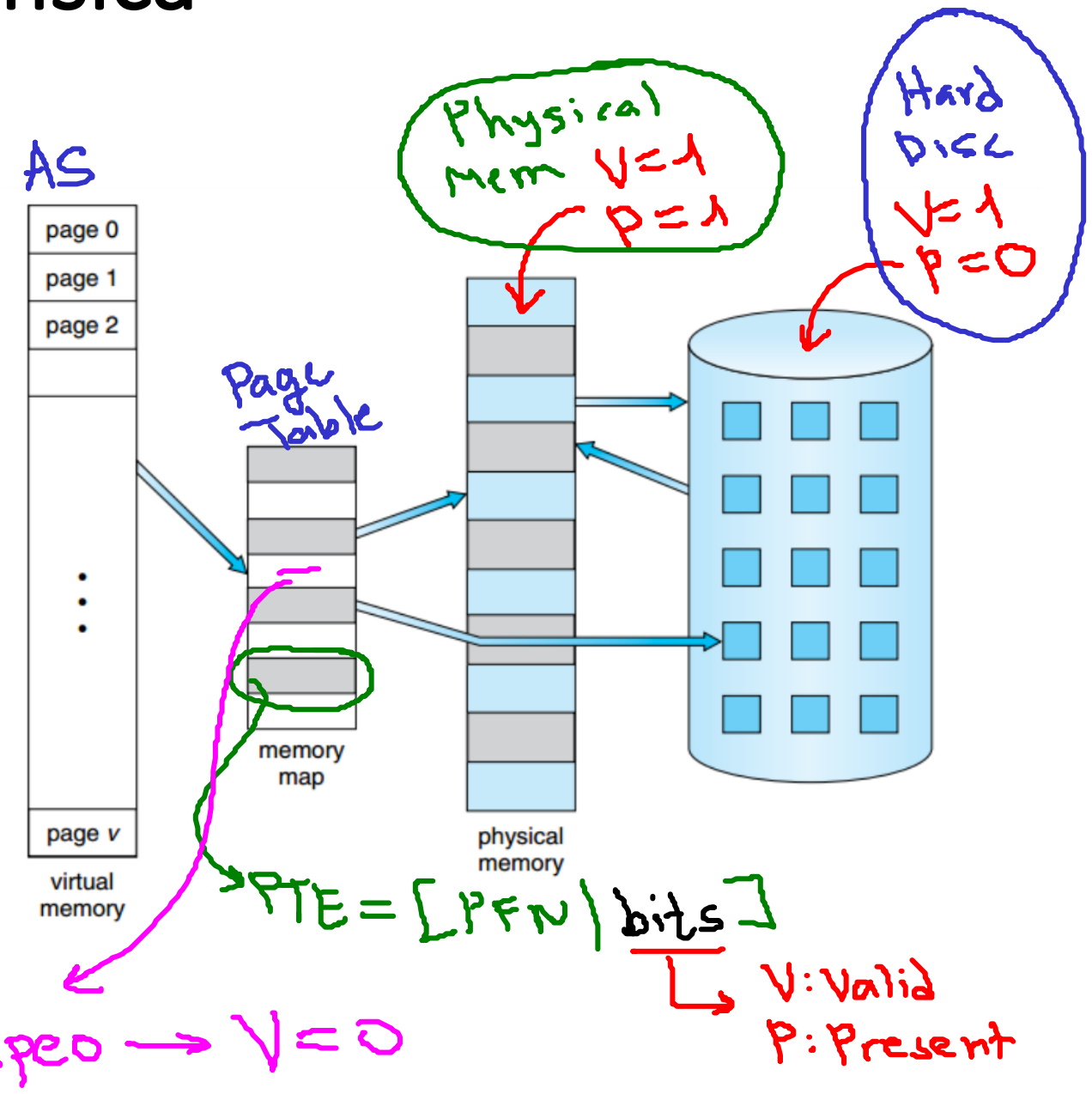
- El programador es quien mueve los fragmentos de código (overlays) dentro y fuera de memoria según sea necesario. Ejemplo: Llamado a una función (**Overlay area ← code + data**)



Más allá de la memoria física

Espacio swap

- Con la llegada de la **multiprogramación**, se llegó a la necesidad de soportar más memoria física de la que se tenía disponible.
- La adición de un espacio swap (espacio de intercambio) le permite al sistema operativo soportar la ilusión de una **memoria virtual muy grande** para procesos concurrentes.
- El **espacio swap** es una región del disco duro empleada para almacenar páginas de memoria.



Más allá de la memoria física

Espacio swap

Pregunta:

¿Cómo el sistema operativo identifica la localización de cada página en el espacio de direcciones?

Respuesta:

Cada página del espacio de direcciones tiene tres posibilidades de mapeo:

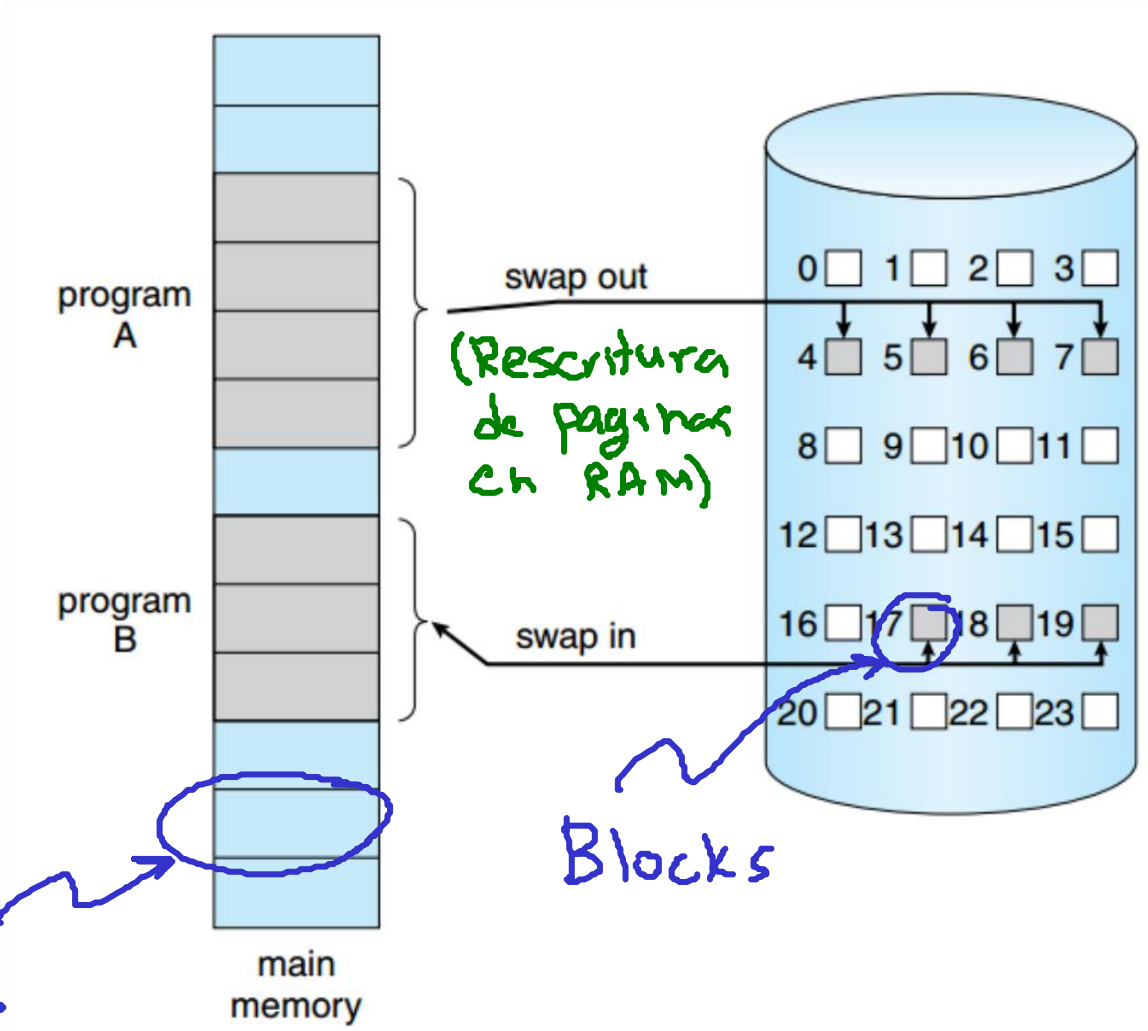
1. Memoria física (V = 1, P = 1)
2. Memoria secundaria (V = 1, P = 0)
3. No se mapea - free (V = 0)

Espacio swap

Introducción

Espacio reservado en el disco duro para almacenar páginas:

- El SO divide el espacio swap en páginas.
- El tamaño del espacio swap determina el número de páginas que el sistema puede utilizar en un momento dado.
- Operaciones de transferencia entre páginas:
  - **Swap out - Page out**
  - **Swap in - Page in**

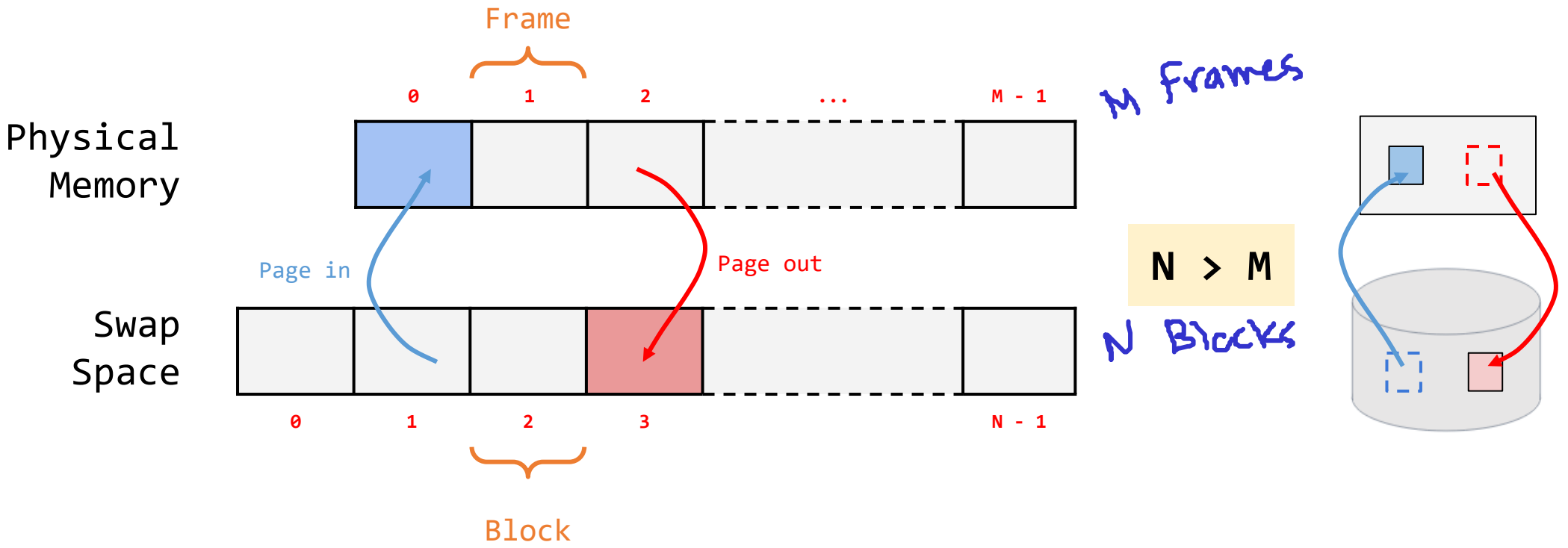


Size (page) = Size (Frame) = Size (block)

Espacio swap

Notación

- **Bloque (block)**: Espacio de memoria (página) en espacio swap.
- **Frame (frame)**: Espacio de memoria (página) en memoria física.



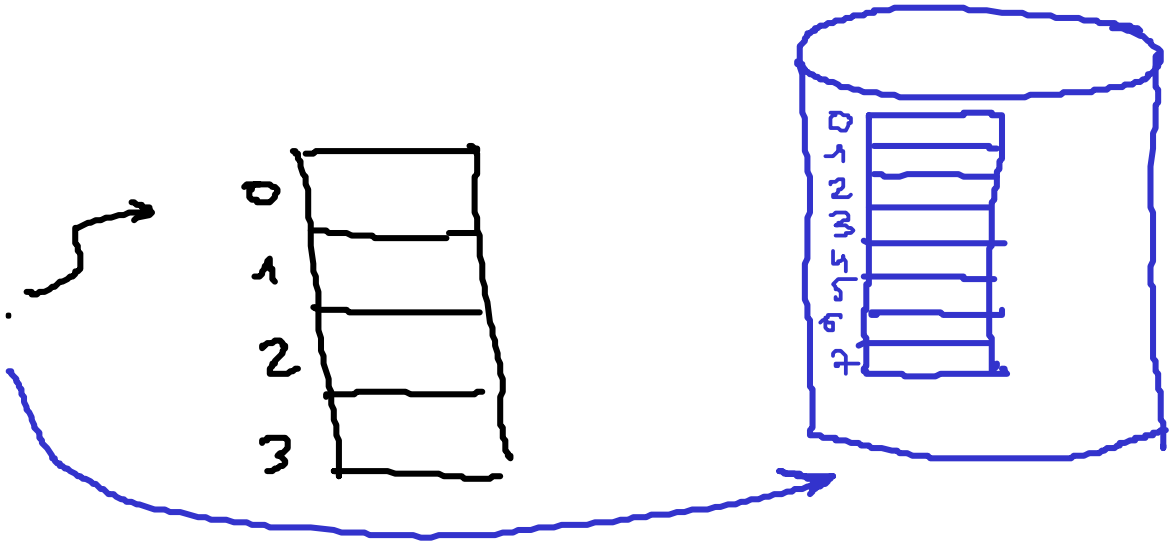
Donde:

- **M:** Número de frames.
- **N:** Número de bloques.

Espacio swap

Ejemplo

- Memoria física de 4 páginas (frames).
- Espacio swap de 8 páginas (blocks).
- Cuatro procesos: P0, P1, P2 y P3.



Las siguientes tablas detallan el mapeo para cada una de las entradas de la tabla de página cuyo bit de validez (V) es 1.

P0 (Proc 0)

VPN	PFN	Block
VPN0	PFN0	---
VPN1	---	Block0
VPN2	---	Block1
...	...	...

P1 (Proc 1)

VPN	PFN	Block
VPN0	---	Block3
VPN1	---	Block4
VPN2	PFN1	---
VPN3	PFN2	---

P2 (Proc 2)

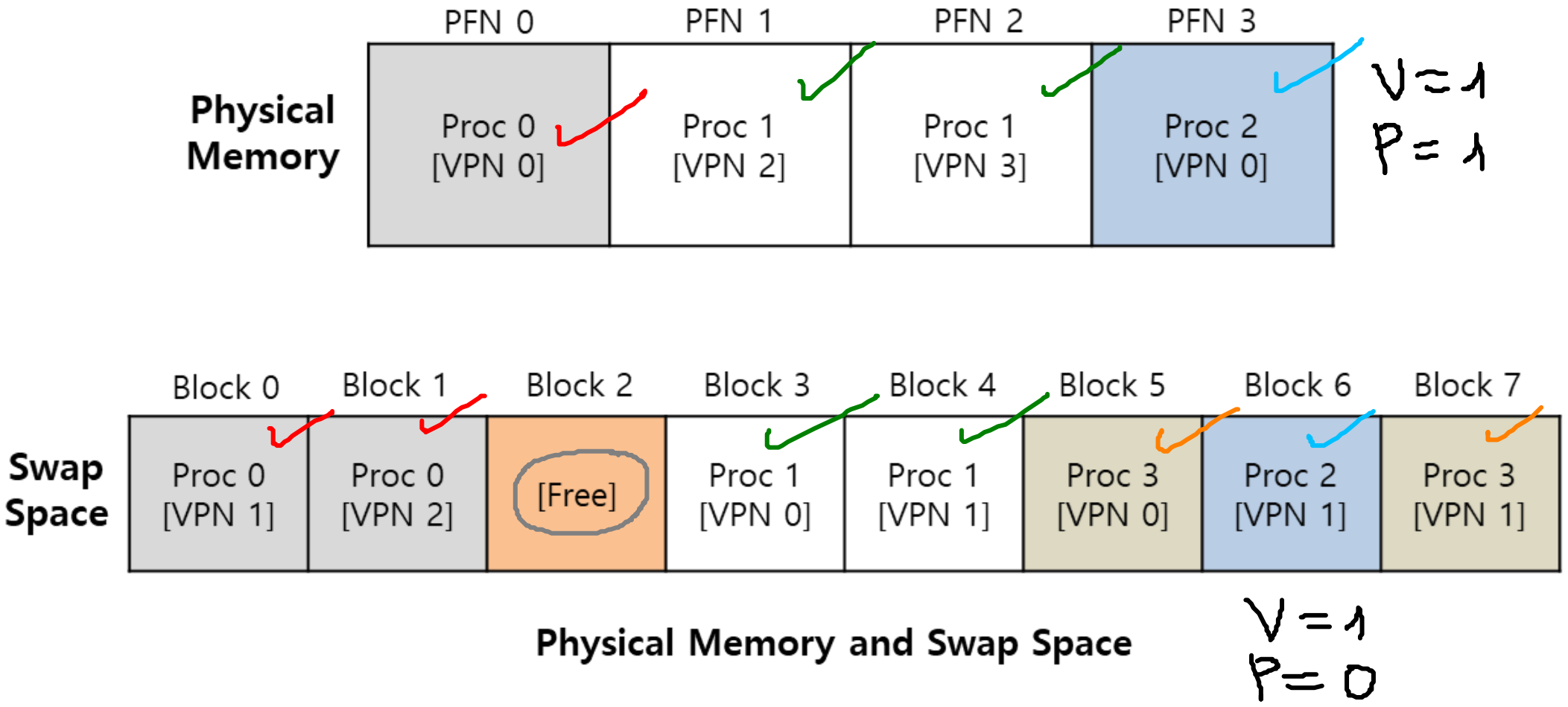
VPN	PFN	Block
VPN0	PFN3	---
VPN1	---	Block6
---	---	---
...	...	...

P3 (Proc 3)

VPN	PFN	Block
VPN0	---	Block5
VPN1	---	Block7
---	---	---
...	...	...

Espacio swap

Ejemplo - Representación

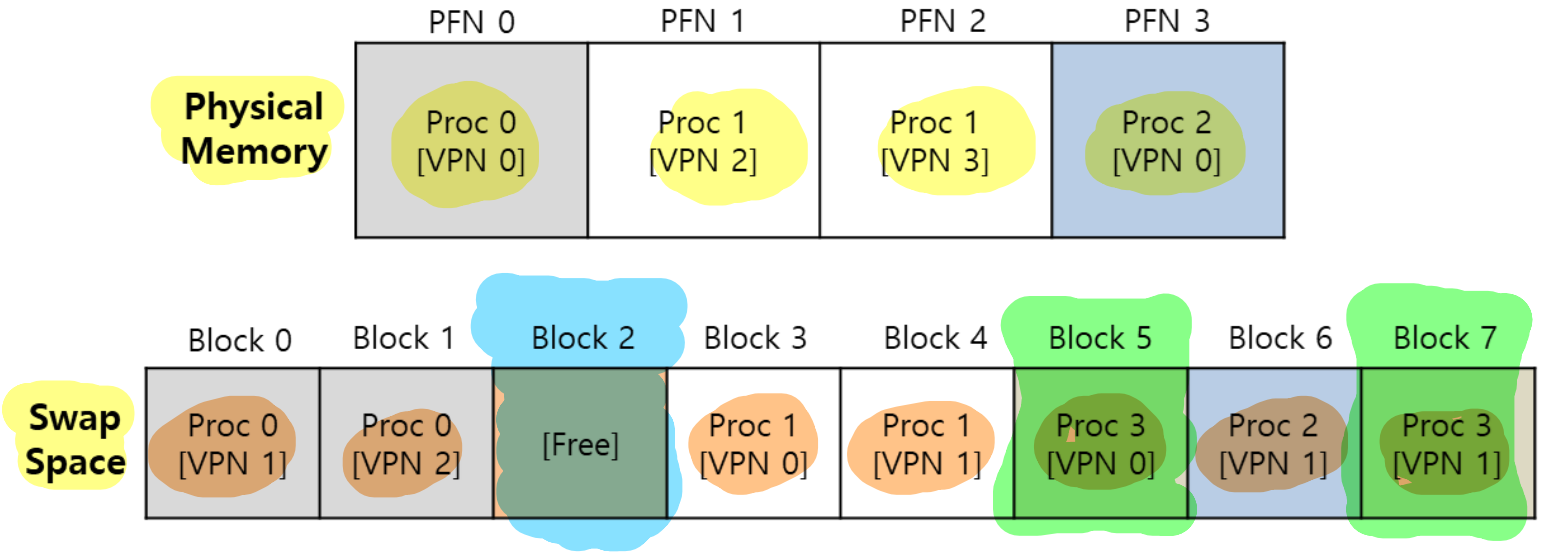


Physical Memory and Swap Space

Espacio swap

Ejemplo - observaciones

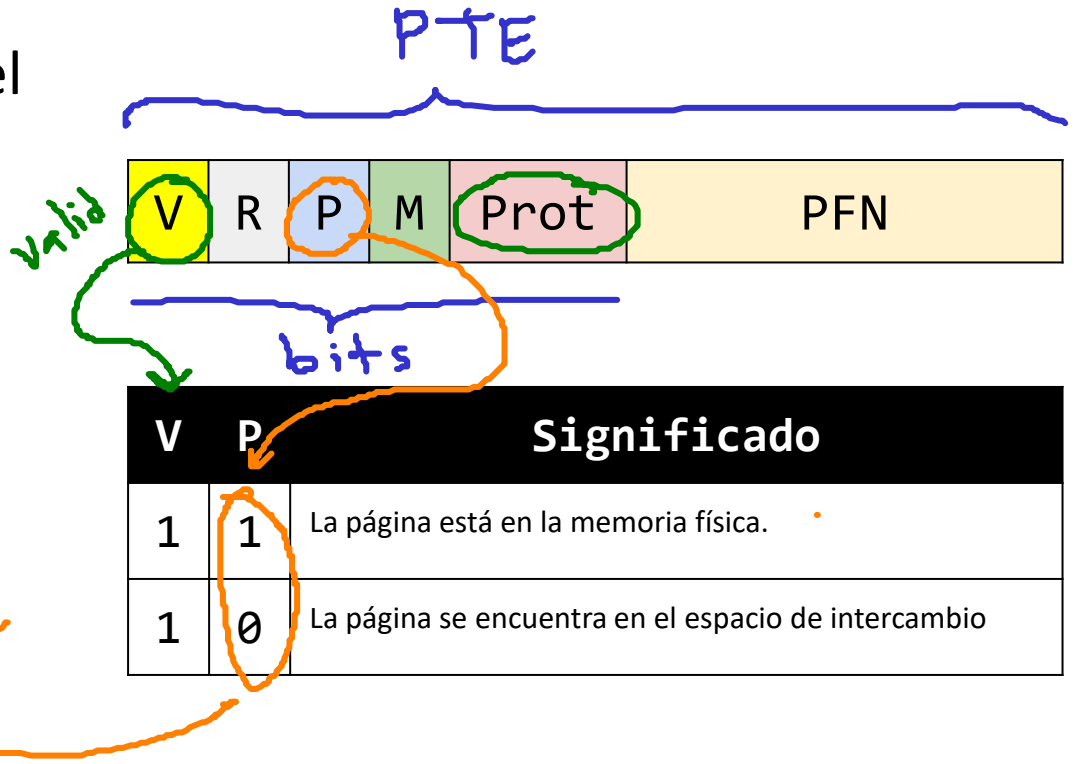
1. P0, P1, P2 y P3 están compartiendo activamente memoria.
2. De las páginas válidas de estos procesos, sólo algunas están en memoria; el resto se encuentra en espacio swap en el disco.
3. Todas las páginas del proceso P3 han sido (**swap out**) sacadas de memoria física y llevadas al disco (Por lo que este proceso no se encuentra actualmente en ejecución).
4. Un bloque de memoria swap (Block 2) permanece libre.



Physical Memory and Swap Space

Bit present

- Se requiere un mecanismo para soportar el **intercambio de páginas** desde (page in) y hacía (page out) el disco duro.
- Cuando el hardware analiza el **PTE**, puede suceder que la pagina requerida no este presente en memoria (a pesar de ser una acceso legal).
- El bit de presencia (P) permite conocer dónde se encuentra la página



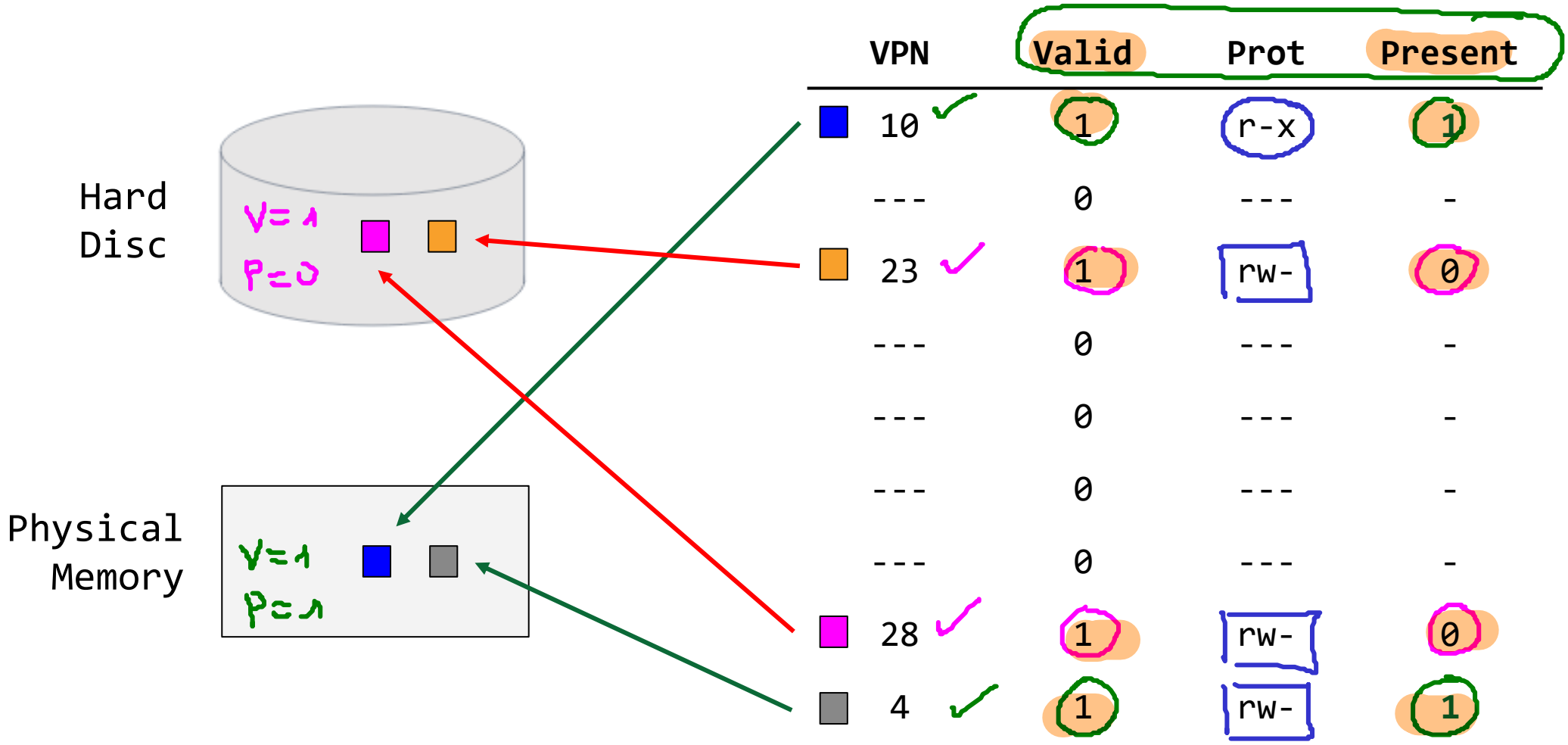
V	P	Significado
1	1	La página está en la memoria física.
1	0	La página se encuentra en el espacio de intercambio

Valor	Significado
1	La página requerida está presente en la memoria física.
0	La página requerida no está presente en la memoria física, pero sí en el disco

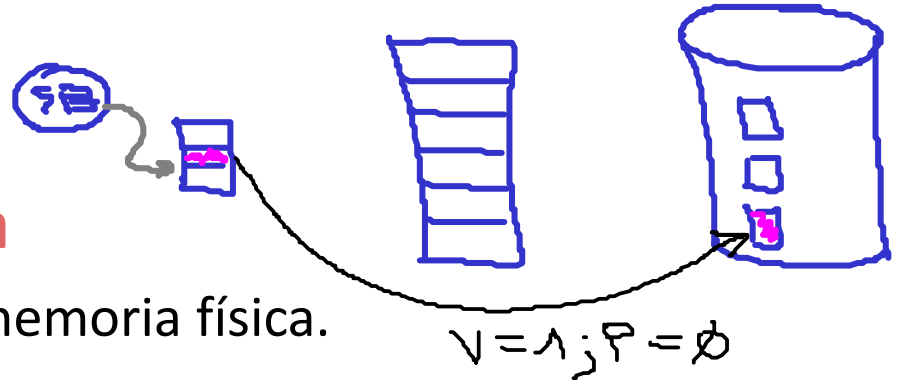
Bit present

Ejemplo

Suponga que se tiene el siguiente escenario de mapeo.

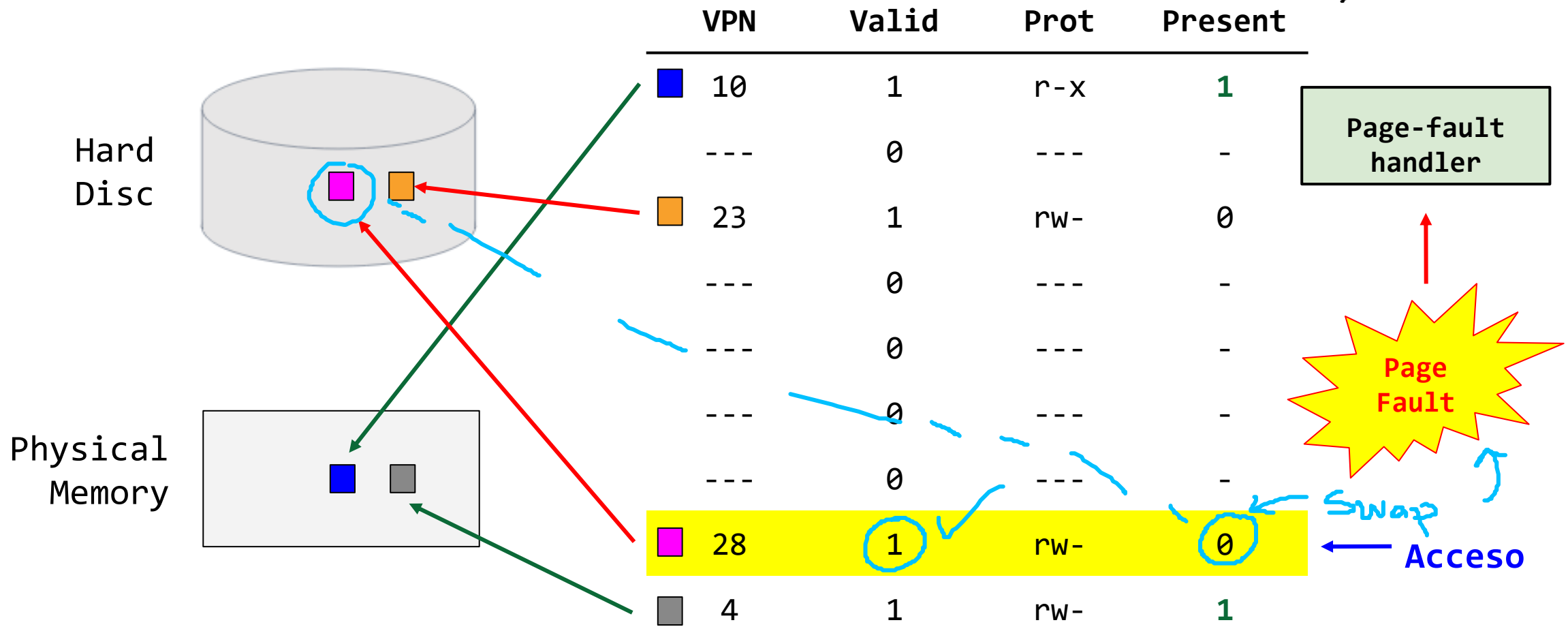


Bit present



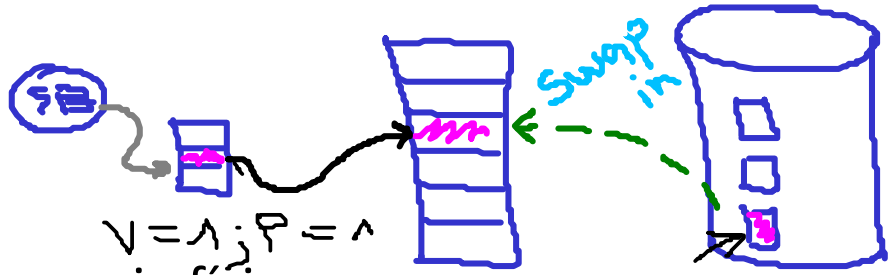
Escenarios con el bit present (P) - Fallo de página

Se da cuando se accede a una página que no se encuentra en memoria física.

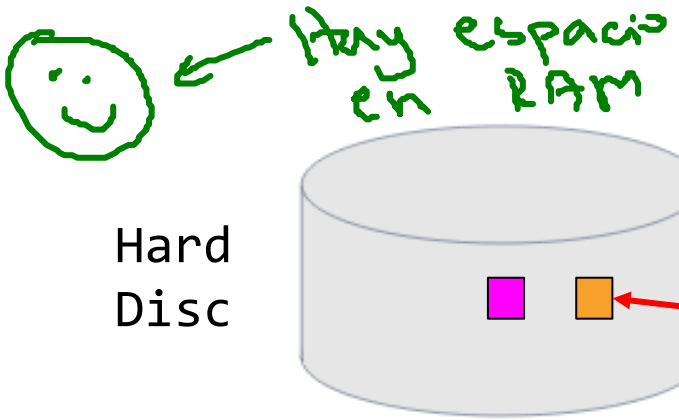


Bit present

Escenarios con el bit present (P) - Fallo de página



Se da cuando se accede a una página que no se encuentra en memoria física.





	VPN	Valid	Prot	Present
■	10	1	r-x	1
	---	0	---	-
■	23	1	rw-	0
	---	0	---	-
	---	0	---	-
	---	0	---	-
	---	0	---	-
■	28	1	rw-	1
■	4	1	rw-	1

Page-fault handler

Memory phys

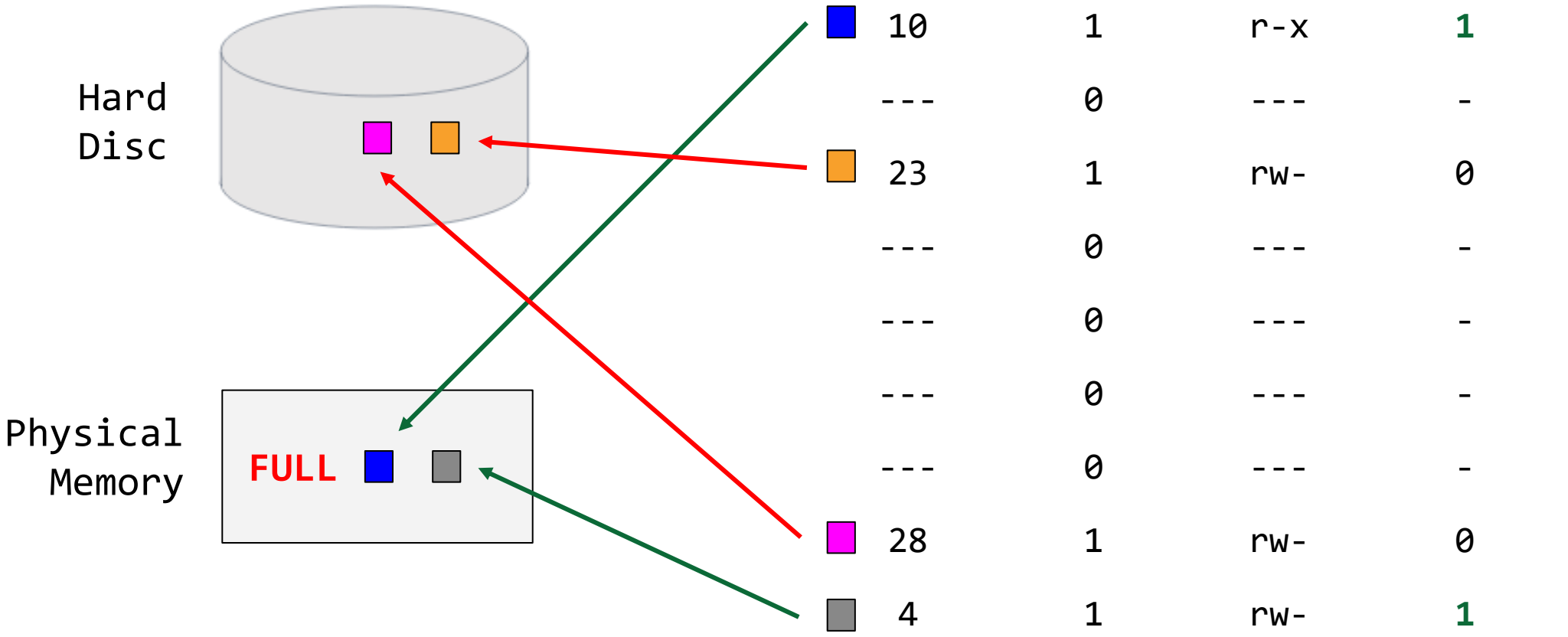
Acceso

Bit present



Escenarios con el bit present (P) - Memoria llena

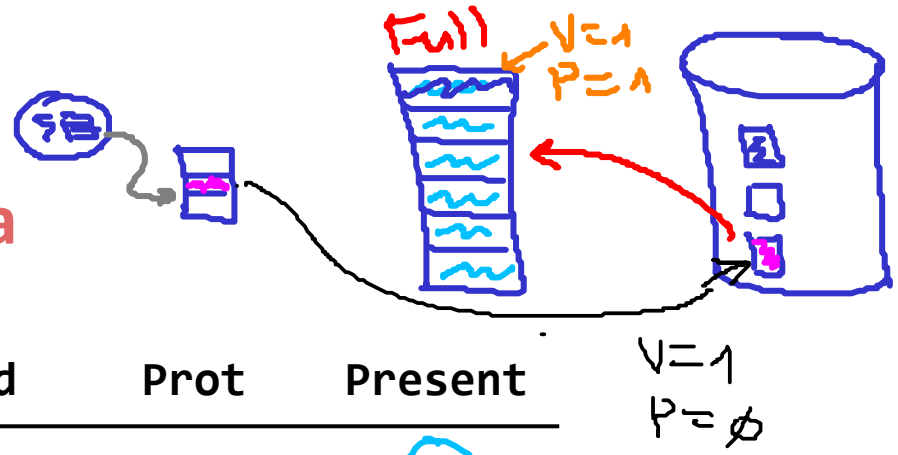
- El sistema operativo retira las páginas de memoria fisica (**page out**) para hacer espacio para las nuevas páginas de memoria requerida usando una **page-replacement policy**.

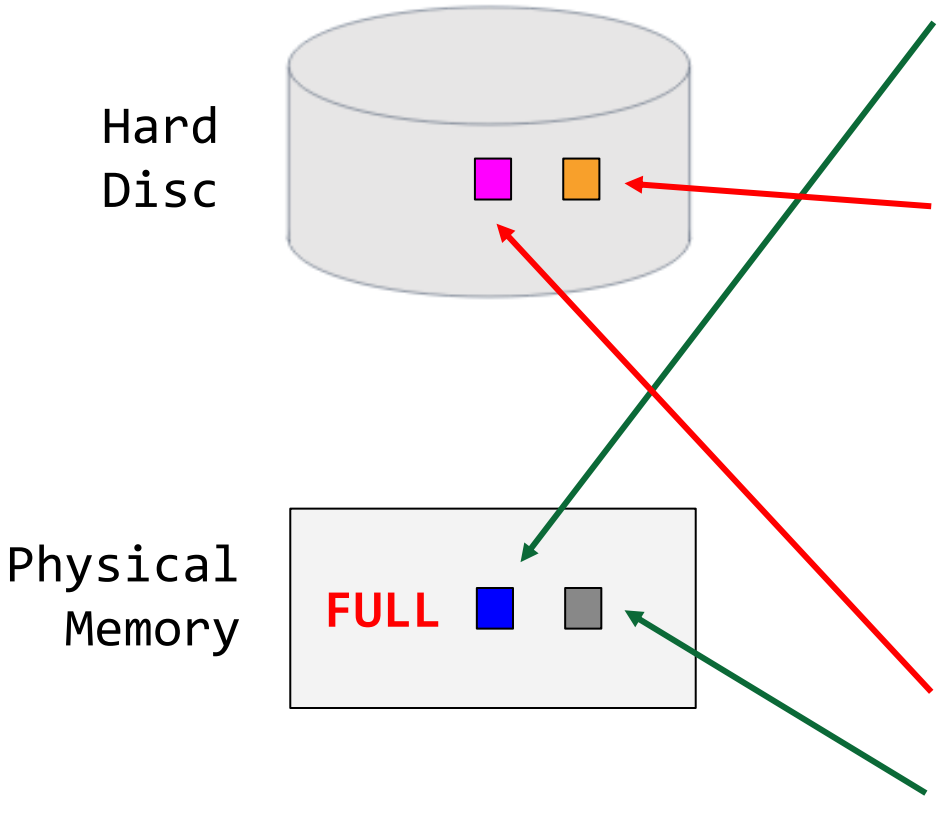


VPN	Valid	Prot	Present
10	1	r-x	1
---	0	---	-
23	1	rw-	0
---	0	---	-
---	0	---	-
---	0	---	-
---	0	---	-
28	1	rw-	0
4	1	rw-	1

Bit present

Escenarios con el bit present (P) - Memoria llena

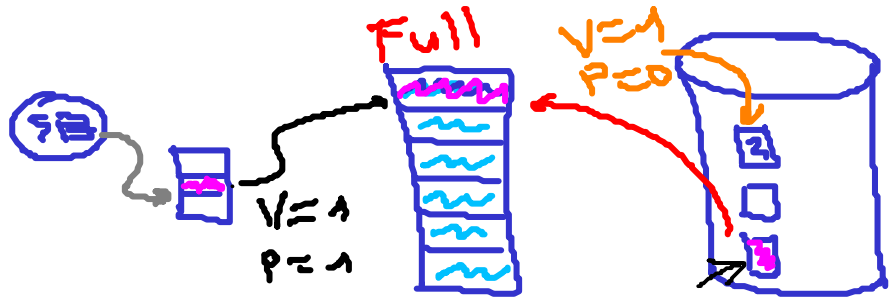


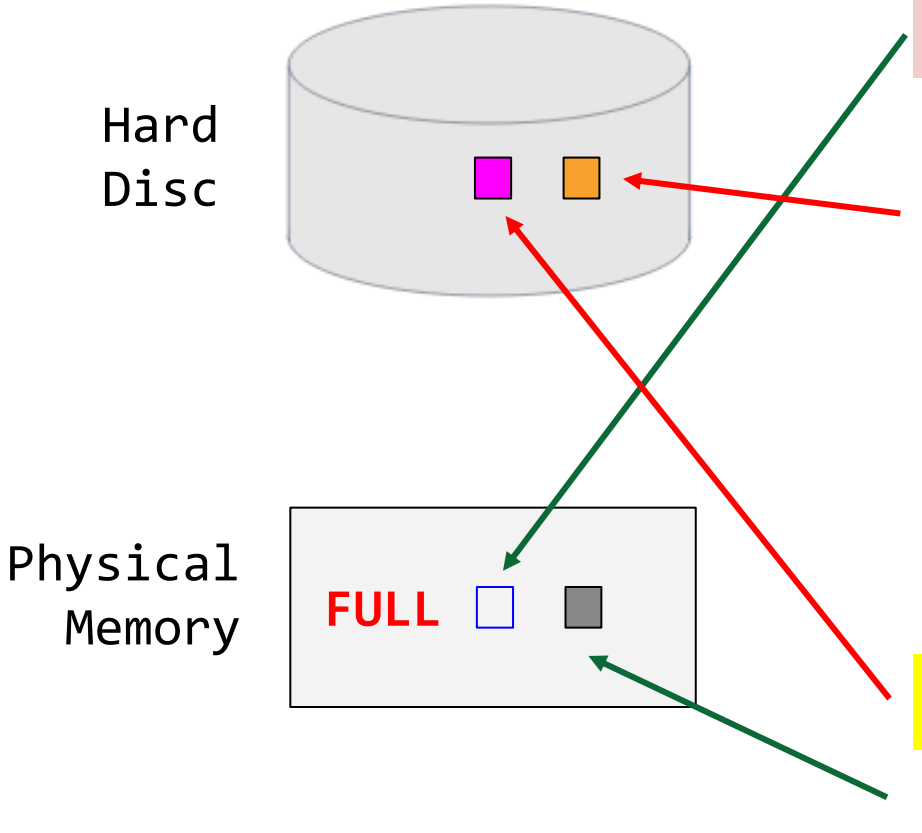


	VPN	Valid	Prot	Present	
■	10	1	r-x	1	
	---	0	---	-	
■	23	1	rw-	0	
	---	0	---	-	
	---	0	---	-	
	---	0	---	-	
	---	0	---	-	
■	28	1	rw-	0	← Acceso
■	4	1	rw-	1	

Bit present

Escenarios con el bit present (P) - Memoria llena

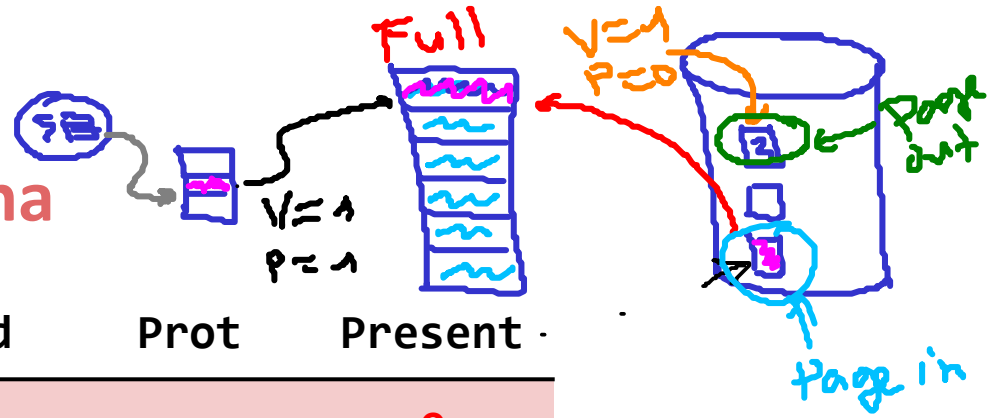




	VPN	Valid	Prot	Present	
■	10	1	r-x	1	← Desalojo (evict)
	---	0	---	-	
■	23	1	rw-	0	
	---	0	---	-	
	---	0	---	-	
	---	0	---	-	
	---	0	---	-	
■	28	1	rw-	0	← Acceso
■	4	1	rw-	1	

Bit present

Escenarios con el bit present (P) - Memoria llena



Hard Disc

Physical Memory

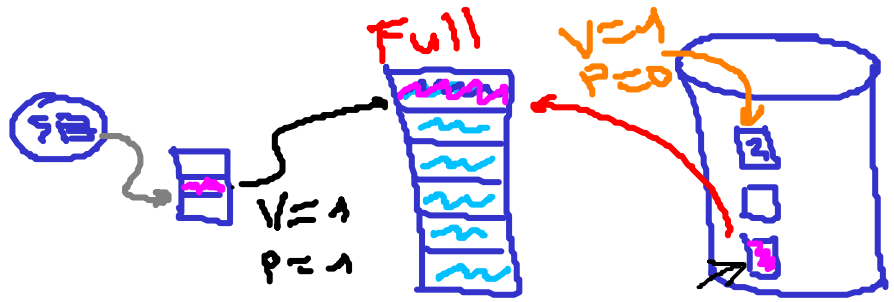
FULL

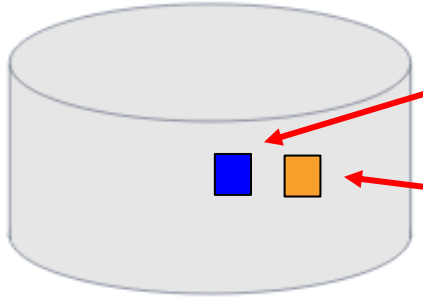
VPN	Valid	Prot	Present	
10	1	r-x	0	← Desalojo (evict)
---	0	---	-	
23	1	rw-	0	
---	0	---	-	
---	0	---	-	
---	0	---	-	
---	0	---	-	
28	1	rw-	0	← Acceso
4	1	rw-	1	

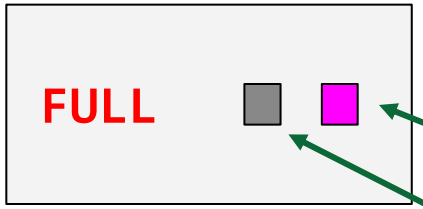
Proceso de “swapping” o “paging” out

Bit present

Escenarios con el bit present (P) - Memoria llena





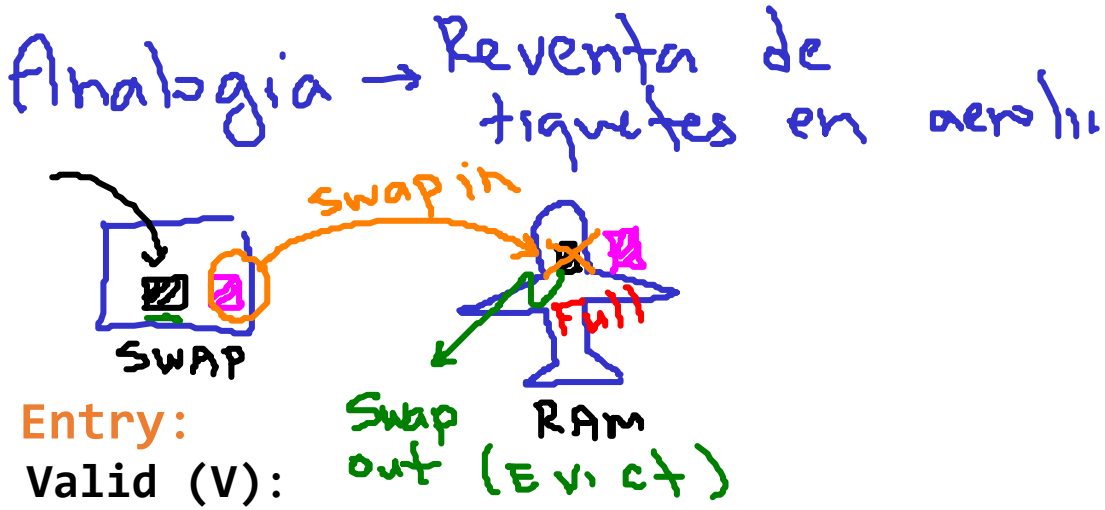


	VPN	Valid	Prot	Present	
■	10	1	r-x	0	← Desalojo (evict)
	---	0	---	-	
■	23	1	rw-	0	
	---	0	---	-	
	---	0	---	-	
	---	0	---	-	
	---	0	---	-	
■	28	1	rw-	1	← Acceso
■	4	1	rw-	1	

Proceso de “swapping” o “paging” out

Traducción de direcciones

Rol de los bits V (Valid) y P (Present)



TLB Entry:

- **Valid (V)**: Dice si la entrada es una traducción válida o no.

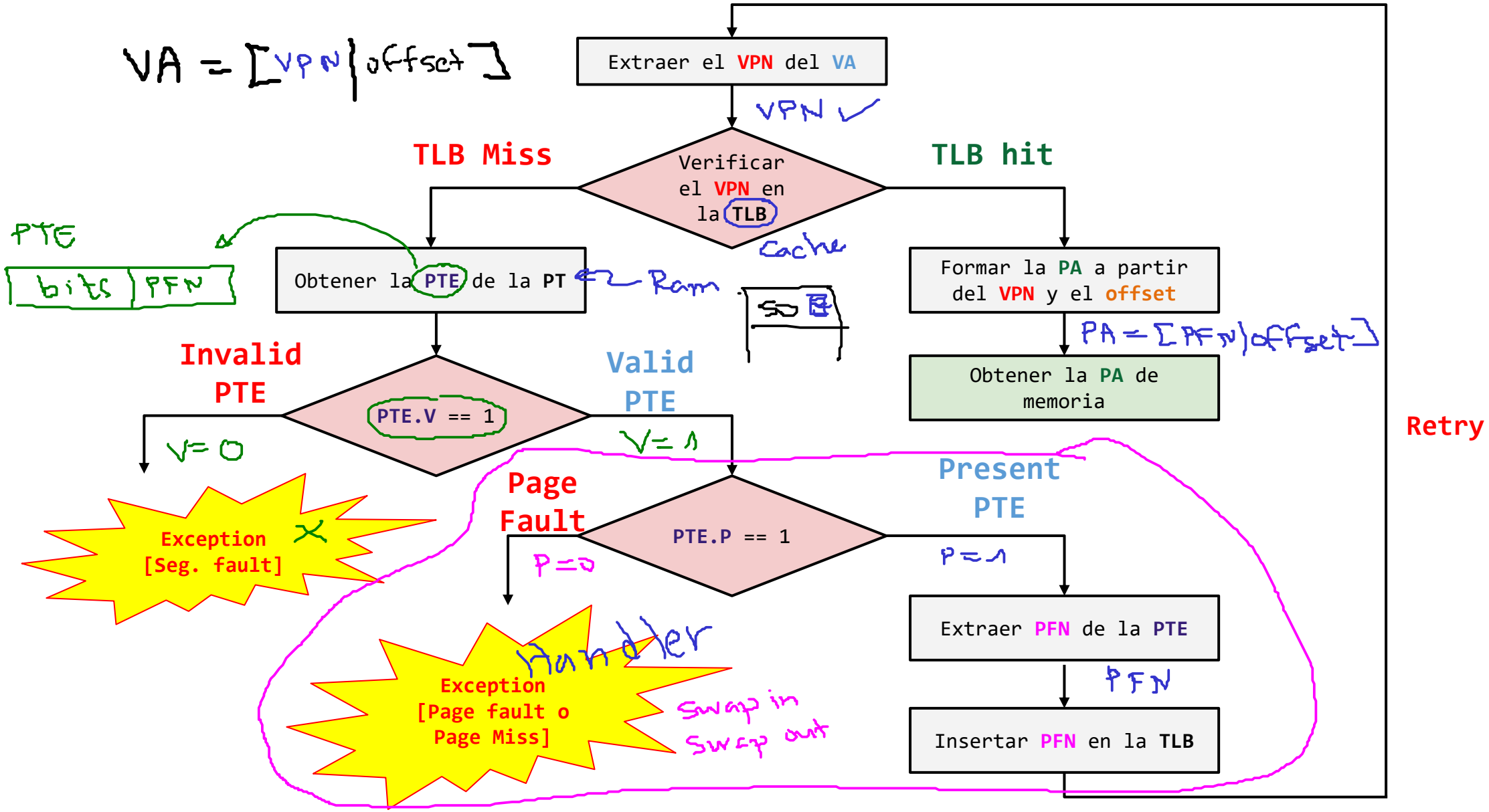
VPN	PFN	Valid	Prot	ASID

PT Entry:

- **Valid (V)**:
  - **1**: El proceso asignó la pagina.
  - **0**: La página no ha sido asignada por el proceso.
- Present (P):
  - **1**: La página se encuentra en memoria física.
  - **0**: La página se encuentra en el disco

PFN	Valid	Prot	Present

Traducción de direcciones



Traducción de direcciones

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True) // TLB Hit
4    if (CanAccess(TlbEntry.ProtectBits) == True)
5      Offset = VirtualAddress & OFFSET_MASK
6      PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7      Register = AccessMemory(PhysAddr)
8    else RaiseException(PROTECTION_FAULT)
9      else // TLB Miss
10       PTEAddr = PTBR + (VPN * sizeof(PTE))
11       PTE = AccessMemory(PTEAddr)
12       if (PTE.Valid == False)
13         RaiseException(SEGMENTATION_FAULT)
14       else
15         if (CanAccess(PTE.ProtectBits) == False)
16           RaiseException(PROTECTION_FAULT)
17         else if (PTE.Present == True)
18           // assuming hardware-managed TLB
19           TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
20           RetryInstruction()
21         else if (PTE.Present == False)
22           RaiseException(PAGE_FAULT)
```



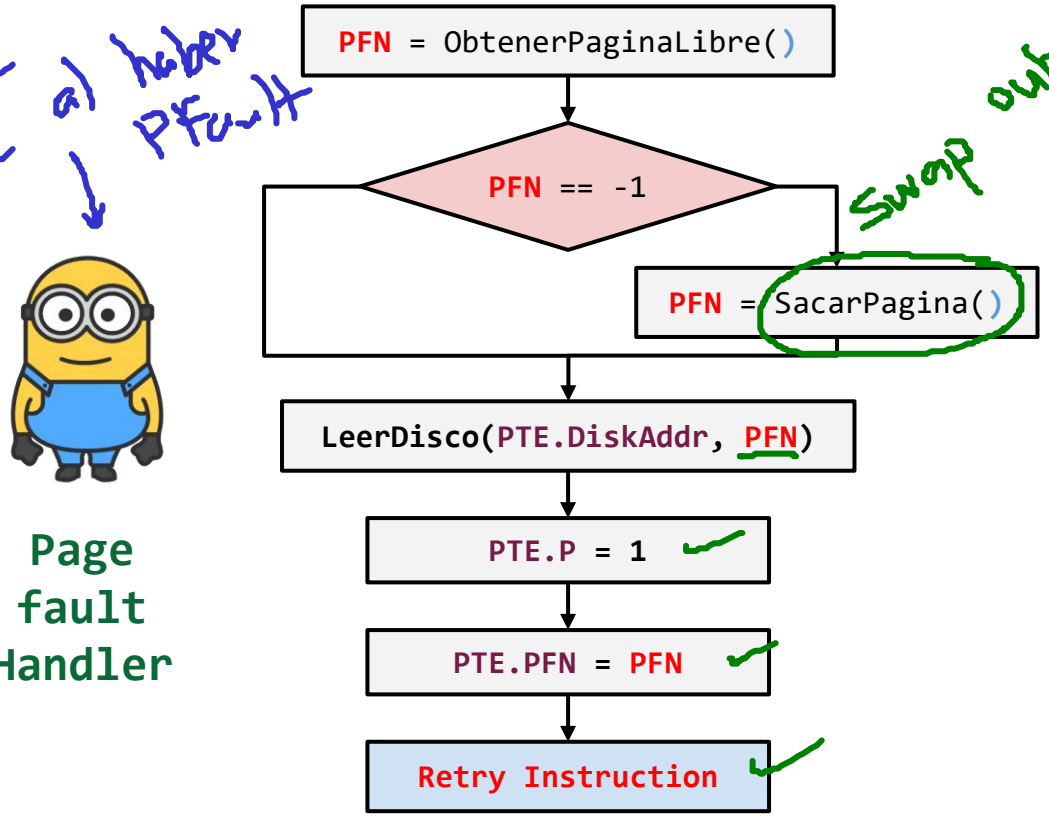
Page-Fault Control Flow Algorithm (Hardware)

Traducción de direcciones

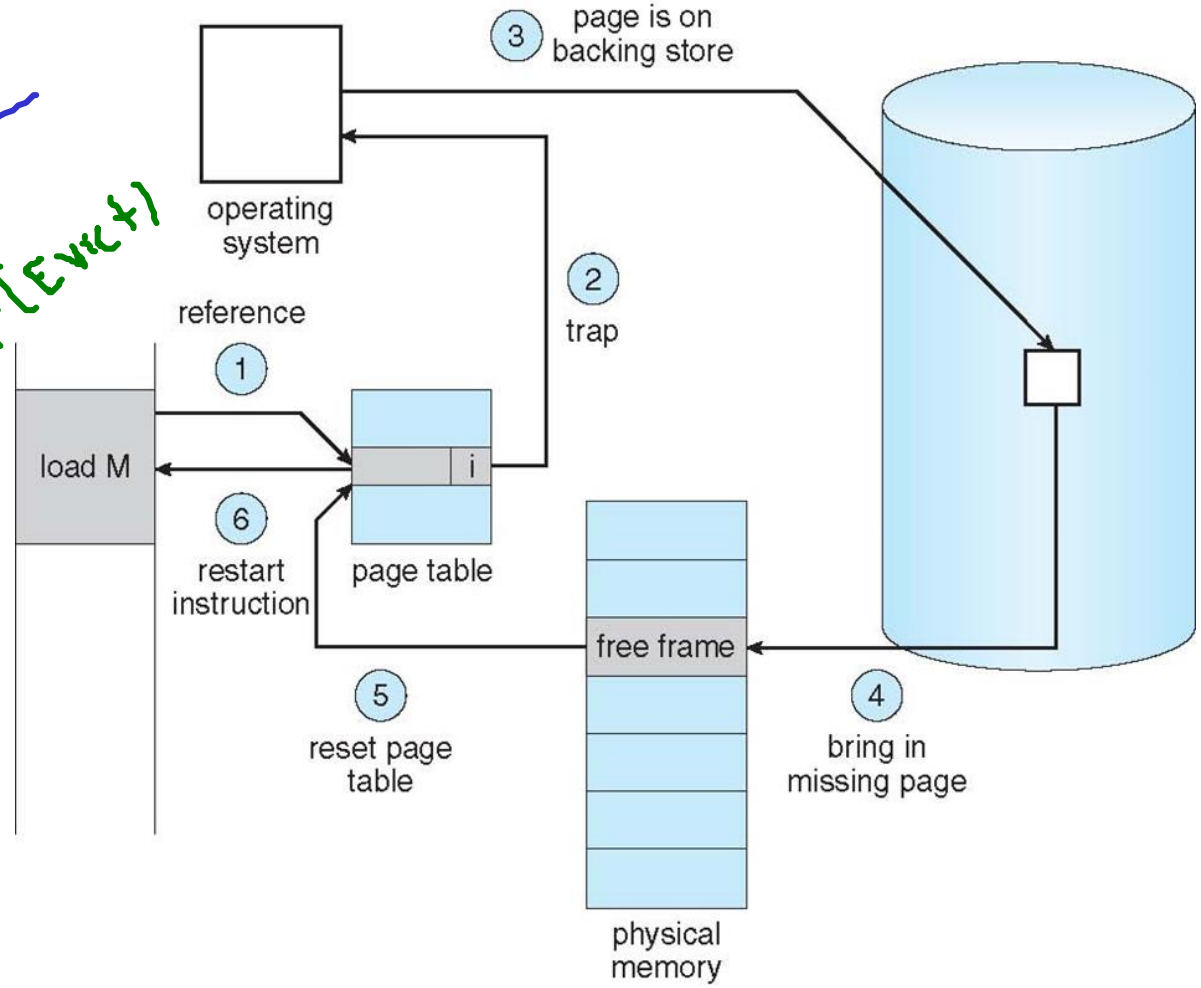
Fallo de página

Cuando ocurre un fallo de página:

- El SO accede a la PTE.
- Obtiene la dirección de la página (en el disco).
- Lanza el requerimiento al disco



Page fault Handler



Traducción de direcciones

Cuando hay un fallo de página

- El sistema operativo debe encontrar el frame (dentro de la memoria física) que será asociado a la página que generó el fallo.
- Si no se encuentra la página en memoria física (frame), es necesario esperar a que se ejecute el algoritmo de reemplazo con el fin de liberar espacio en memoria física sacando algunas páginas de esta.



Page fault Handler

```
1  PFN = FindFreePhysicalPage()
2  if (PFN == -1)                  // no free page found
3    PFN = EvictPage()             // run replacement algorithm
4  DiskRead(PTE.DiskAddr, PFN)     // sleep (waiting for I/O)
5  PTE.present = True              // update page table with present
6  PTE.PFN = PFN                   // bit and translation (PFN)
7  RetryInstruction()              // retry instruction
```

Page-Fault Control Flow Algorithm (Software)

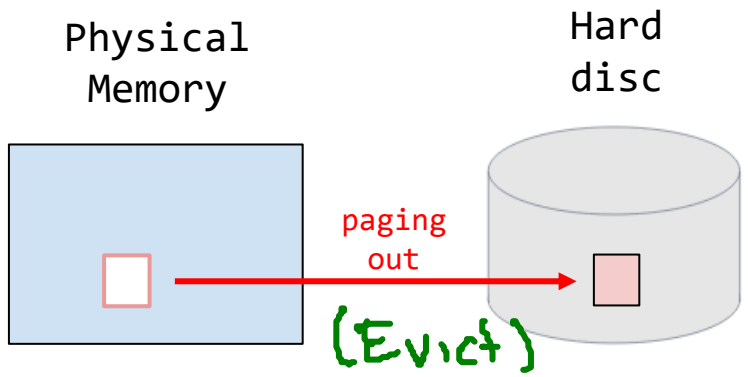
Reemplazo

¿Cual página sacar? → **Política de reemplazo (page-replacement policy)**

La política de reemplazo es el proceso de selección de una pagina para ser retirada de memoria física.

```
1  PFN = FindFreePhysicalPage()
2  if (PFN == -1)                  // no free page found
3    PFN = EvictPage()             // run replacement algorithm
4  DiskRead(PTE.DiskAddr, PFN)   // sleep (waiting for I/O)
5  PTE.present = True            // update page table with present
6  PTE.PFN = PFN                 // bit and translation (PFN)
7  RetryInstruction()            // retry instruction
```

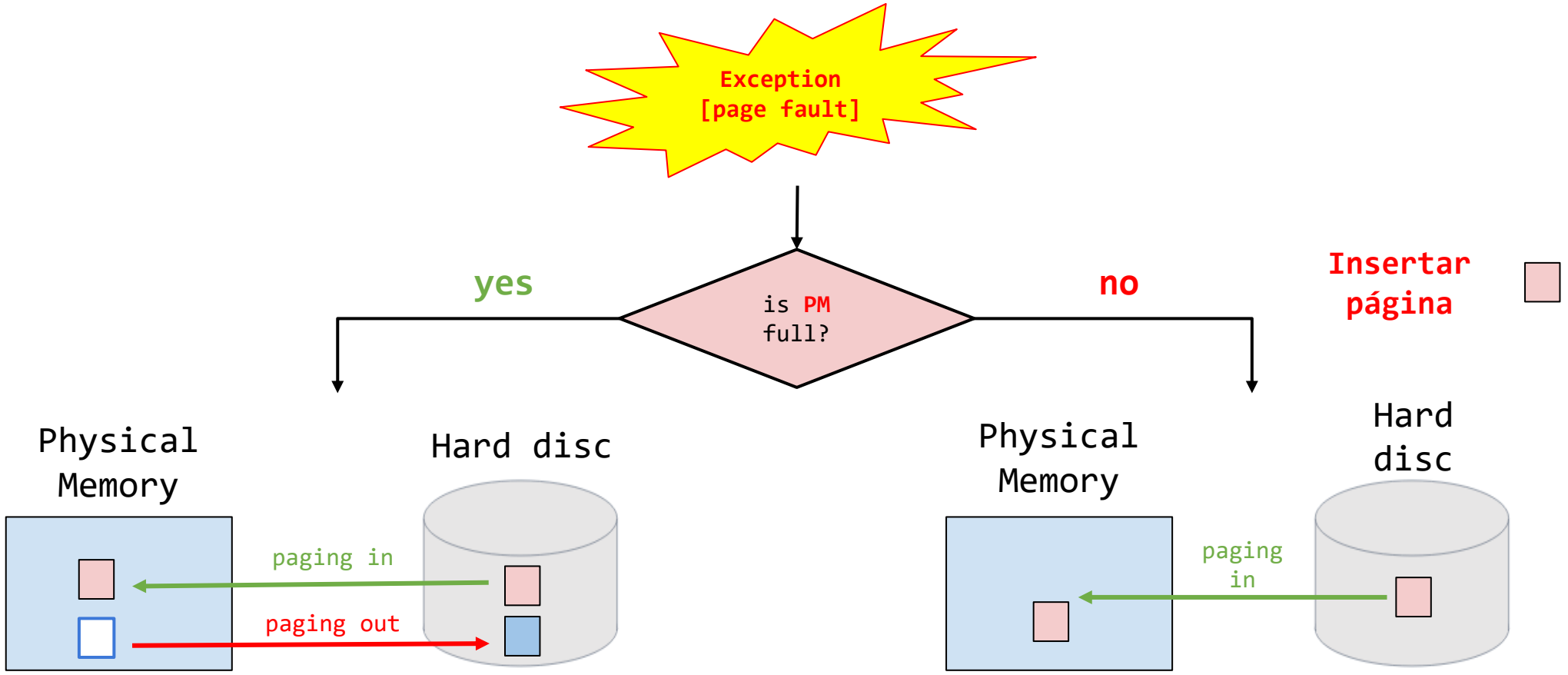
Next class



Reemplazo

Proceso inicial (el que habíamos dicho al principio)

Cuando la memoria está llena, el sistema operativo retira (page out) páginas de memoria (frames) para hacer espacio para nuevas páginas. Sin embargo, esta aproximación **no es realista**.

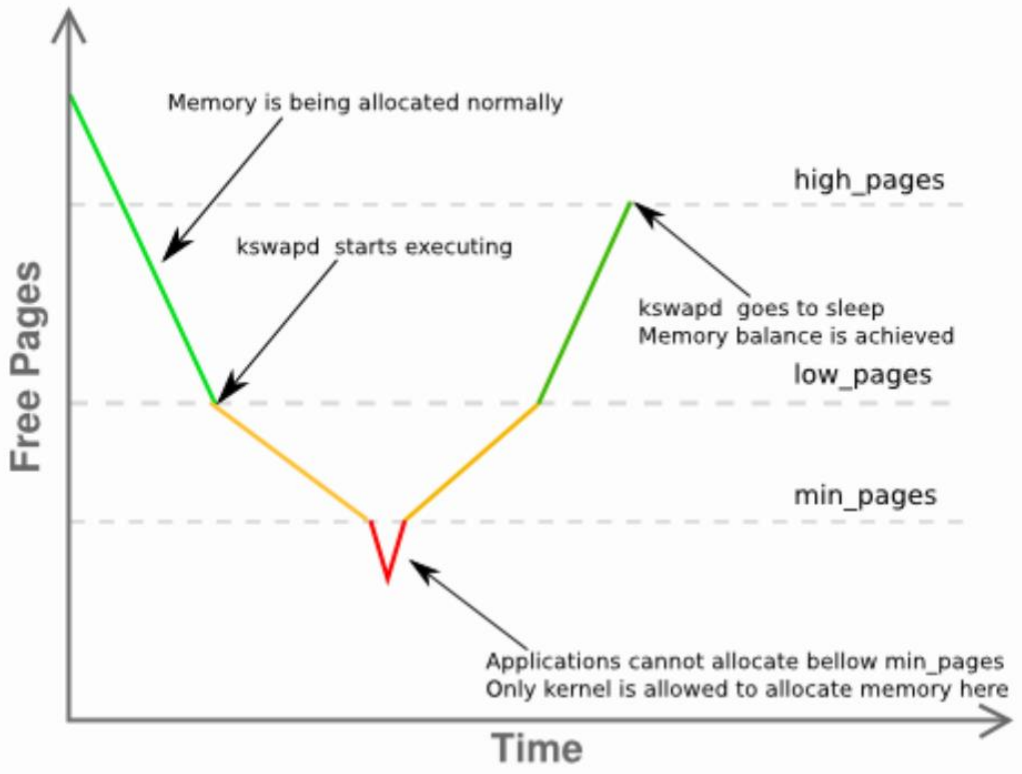


Reemplazo

Video del Prof. Danny.

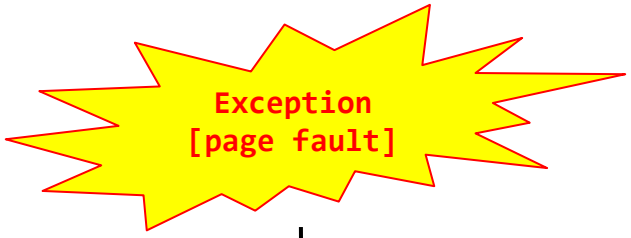
Proceso real

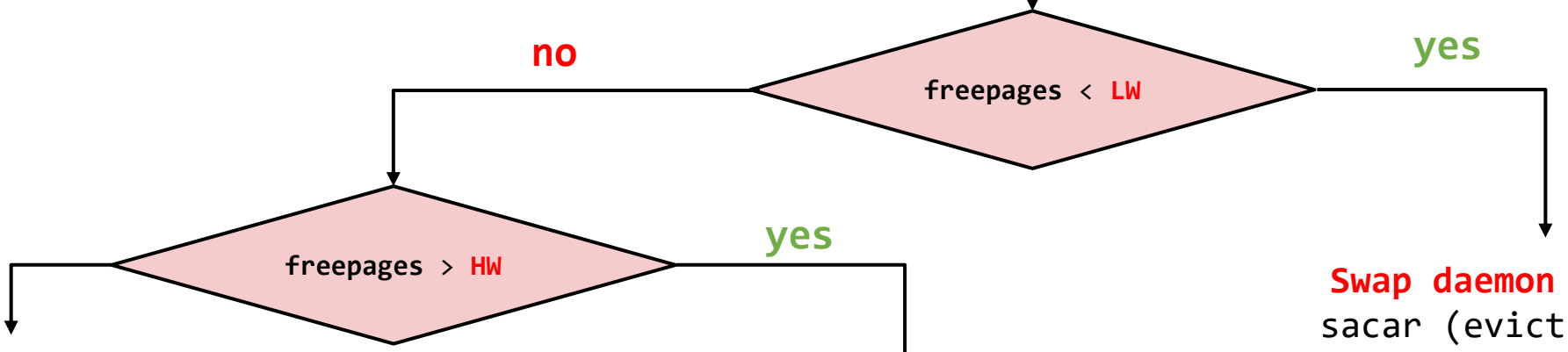
- El sistema operativo busca mantener una pequeña porción de memoria libre.
- Se manejan dos umbrales:
  - HW (High watermark).
  - LW (Low watermark).
- Un **background thread** llamado **swap daemon (page daemon)** es responsable de liberar memoria.



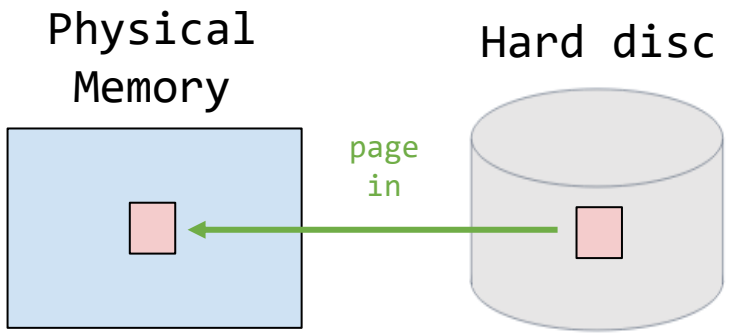
Reemplazo

Proceso real





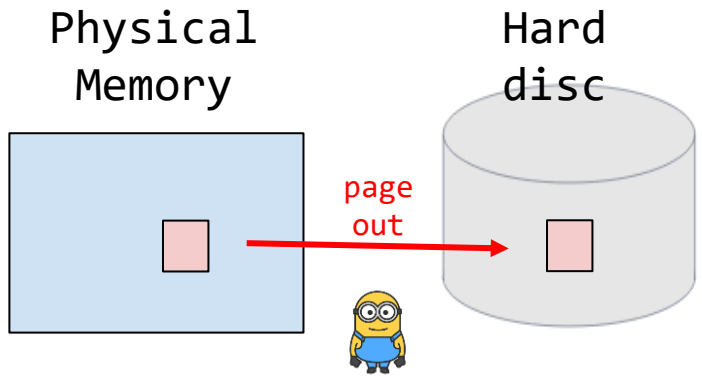
La memoria está siendo asignada **normalmente**.



Swap daemon va a dormir.



Swap daemon empieza a sacar (evict) páginas.



Referencias

- **Operating Systems Three Easy Pieces** (website: Capítulo 21)
- Operating System Concepts - Tenth Edition (website)
- Operating Systems Notes (link)
- http://web.eecs.utk.edu/~mbeck/classes/cs560/560/oldtests/t2/2003/Answers.html
- https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/1-Overview.md
- https://myaut.github.io/dtrace-stap-book/kernel/proc.html
- https://myaut.github.io/dtrace-stap-book/index.html
- https://www.technologyuk.net/computing/computer-software/operating-systems/operating-system-utilities.shtml
- http://www.it.uu.se/education/course/homepage/os/vt18/module-4/simple-threads/
- https://courses.cs.duke.edu/spring19/compsci590.1/slides/
- http://csl.snu.ac.kr/courses/4190.307/2020-1/
- http://pages.cs.wisc.edu/~bart/537/lecturenotes/titlepage.html
- https://www.scs.stanford.edu/22wi-cs212/

Referencias

- https://flint.cs.yale.edu/cs421/papers/x86-asm/asm.html
- https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/5-Memory-Management.md
- https://applied-programming.github.io/Operating-Systems-Notes/5-Memory-Management/
- https://csapp.cs.cmu.edu/
- https://applied-programming.github.io/Operating-Systems-Notes/5-Memory-Management/
- http://www.cs.cmu.edu/~213/
- https://web.eecs.umich.edu/~akamil/teaching/sp04/
- https://courses.engr.illinois.edu/cs241/sp2014/
- https://emeryberger.com/teaching/grad-systems/
- https://oslab.kaist.ac.kr/courseware/
- https://oslab.kaist.ac.kr/ee488-fall-2020/